



Specifica Tecnica

02/04/2026



Revisioni

Ver.	Autore	Modifiche	Data	Verifica
1.0	spadiliero ruben	approvazione documento per pb	14/05/2026	Gabriele Disa
0.10	Gabriele Disa	Aggiornamento sistema di deploy (sezione 3.2)	14/05/2026	Stefano Maso
0.9	Niccolò Feltrin	Aggiunta introduzione al frontend (sezione 3.1.2)	14/05/2026	Besnik Murtezan
0.8	Niccolò Feltrin	Aggiornata sezione architettura logica frontend (sezione 3.1.2)	12/05/2026	Spadiliero Ruben
0.7	Besnik Murtezan	Aggiornate descrizioni tecnologie frontend	11/05/2026	Niccolò Feltrin
0.6	Stefano Maso	Modifica sezione LLM	24/04/2026	Giuliano Banchieri
0.5	Giuliano Banchieri	Stesura sezione architettura di deployment e organizzazione container	21/04/2026	Spadiliero Ruben
0.4	Stefano Maso	Aggiunte sottosezioni per l'importazione	10/04/2026	Besnik Murtezan
0.3	Stefano Maso	Stesura sezione 3.1.1.2 e sottosezioni	09/04/2026	Besnik Murtezan
0.2	Stefano Maso	Stesura sezione 3.1, 3.1.1 e sottosezioni relative all'architettura del backend e relativi diagrammi delle classi	08/04/2026	Besnik Murtezan
0.1	Giuliano Banchieri	Prima stesura: struttura del documento, tecnologie, riferimenti, introduzione	02/04/2026	Niccolò Feltrin



Indice

1	Introduzione	7
1.1	Scopo del documento	7
1.2	Scopo del prodotto	7
1.3	Riferimenti	7
1.3.1	Riferimenti normativi	7
1.3.2	Riferimenti informativi	7
2	Tecnologie	9
2.1	Tecnologie di codifica	9
2.1.1	TypeScript	9
2.1.2	Vue.js	9
2.1.3	Marked.js	10
2.1.4	CSS	10
2.1.5	Tailwind CSS	11
2.1.6	HTML	11
2.1.7	PHP	11
2.1.8	Laravel	11
2.2	Tecnologie di verifica	11
2.2.1	PHPUnit	11
2.3	Tecnologie per la persistenza dei dati (OPT)	11
2.3.1	PostgreSQL	11
2.4	Tecnologie di supporto	12
2.4.1	Vite	12
2.4.2	Composer	12
2.4.3	nginx	12
2.4.4	PHP-FPM	12
2.4.5	Docker	12
2.4.6	Docker Compose	12
2.5	Modelli LLM esterni	12
3	Architettura	13
3.1	Architettura logica	13
3.1.1	Backend	13
3.1.1.1	Gestione note e Esportazione	14
3.1.1.1.1	NoteController	14
3.1.1.1.2	NoteService	14
3.1.1.1.3	NoteRepositoryInterface	15
3.1.1.1.4	NoteRepository	15
3.1.1.1.5	Note	15
3.1.1.1.6	ImportController	15
3.1.1.1.7	ImportService	16
3.1.1.1.8	ImportStrategyFactory	16
3.1.1.1.9	ImportStrategyInterface	16
3.1.1.1.10	MdImport	16
3.1.1.1.11	ExportController	16
3.1.1.1.12	ExportService	16
3.1.1.1.13	ExportStrategyFactory	17
3.1.1.1.14	ExportStrategyInterface	17
3.1.1.1.15	MdExport	17
3.1.1.1.16	HtmlExport	17
3.1.1.1.17	PdfExport	17
3.1.1.1.18	EditorContentFormatter	17
3.1.1.2	Interazione con LLM ^G	18



3.1.1.2.1	LlmController	18
3.1.1.2.2	LlmService	19
3.1.1.2.3	LlmStrategyFactory	19
3.1.1.2.4	LlmProcessorResponse	19
3.1.1.2.5	LlmStrategyInterface	19
3.1.1.2.6	LlmContext	19
3.1.1.2.7	Strategy concrete	20
3.1.2	Frontend	21
3.1.2.1	Root Component	22
3.1.2.1.1	App	22
3.1.2.1.2	AppLayout	22
3.1.2.1.3	RouterView	22
3.1.2.1.4	AppSidebar	23
3.1.2.1.5	NavButton	23
3.1.2.1.6	NavItem	23
3.1.2.1.7	useTheme	23
3.1.2.1.8	ActionModal	24
3.1.2.1.9	BaseModal	24
3.1.2.1.10	useModals	24
3.1.2.1.11	ToastList	25
3.1.2.1.12	ActionToast	25
3.1.2.1.13	useToast	25
3.1.2.2	Comunicazione con backend	26
3.1.2.2.1	Note (type)	26
3.1.2.2.2	NoteWithContent (type)	26
3.1.2.2.3	NoteAPI (type)	26
3.1.2.2.4	apiClient	26
3.1.2.2.5	noteService	27
3.1.2.2.6	aiService	27
3.1.2.2.7	serviceHandler	28
3.1.2.3	Composables Note & AI	28
3.1.2.3.1	useNotes	28
3.1.2.3.2	NoteNotUpdatedError	29
3.1.2.3.3	useAi	30
3.1.2.4	Pagina di archivio note	31
3.1.2.4.1	NoteArchivePage	31
3.1.2.4.2	NoteArchiveCard	31
3.1.2.4.3	ContextAction (type)	31
3.1.2.4.4	useContextMenu	32
3.1.2.4.5	ContextMenu	32
3.1.2.5	Pagina di editor note	32
3.1.2.5.1	NoteEditorPage	32
3.1.2.5.2	useNoteEditor	33
3.1.2.5.3	useNoteEditorUI	33
3.1.2.5.4	AsideAI	34
3.1.2.5.5	NoteEditorHeader	35
3.1.2.5.6	NoteEditorActions	35
3.1.2.5.7	NoteEditorToolbar	36
3.1.2.5.8	NoteEditorContent	36
3.2	Architettura di deployment	36
3.2.1	Organizzazione del container	37
3.2.2	Persistenza dei dati	37
3.2.3	Build degli asset frontend	38
3.2.4	Configurazione e avvio	38
3.2.5	Container non presenti nel deploy finale	38



4 Stato dei requisiti funzionali

39



Indice delle immagini

1	Diagramma delle classi per Gestione note - Esportazione - Importazione	14
2	Diagramma delle classi per Interazione con LLM	18
3	Diagramma UML gerarchia principale frontend	22
4	Diagramma UML componente AppSidebar	23
5	Diagramma UML componente ActionModal	24
6	Diagramma UML componente ToastList	25
7	Diagramma UML struttura comunicazione con backend	26
8	Diagramma UML composable useNotes	28
9	Diagramma UML composable useAi	30
10	Diagramma UML pagina di archivio	31
11	Diagramma UML pagina di editor	32

1 Introduzione

1.1 Scopo del documento

Il presente documento contiene informazioni dettagliate sulle tecnologie utilizzate e sulle scelte progettuali del gruppo Tool-8 per la realizzazione del prodotto **Second Brain** su richiesta dell'azienda **Zucchetti S.p.A.**

Sono fornite descrizioni delle tecnologie scelte per backend, frontend e di supporto, i diagrammi delle classi che rappresentano in modo dettagliato l'architettura logica e una tabella di tracciamento dei requisiti soddisfatti.

1.2 Scopo del prodotto

"Estendere il note-taking con i Large Language Models."

L'obiettivo del capitolato^G C6 **"Second Brain"** è lo sviluppo^G di un applicativo che fornisca all'utente un editor di file MD con funzionalità^G AI che semplifichino la scrittura tramite funzionalità^G come: riassunto, traduzione, critica, correzione, e generazione del testo.

L'utente avrà la possibilità di utilizzare queste funzionalità^G anche tramite interrogazioni in linguaggio naturale che verranno processate da un LLM^G contattato dal sistema.

Il software sarà sviluppato sotto forma di applicazione web dove l'utente potrà creare, salvare, eliminare e modificare le sue note.

1.3 Riferimenti

1.3.1 Riferimenti normativi

- Capitolato^G C6 - Progetto^G Second Brain:
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>
- Regolamento progetto^G didattico:
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>
- Norme di Progetto^G v1.1
- Analisi dei Requisiti^G v1.5

1.3.2 Riferimenti informativi

- Verbali
- Glossario v1.0
- Lezioni del corso di *Ingegneria del software*^G aggiornate al 02/04/2026:
 - *"Progettazione^G software (T6)"*:
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/T06.pdf>
 - *"Progettazione^G e programmazione: Diagrammi delle classi (UML)"*:
<https://www.math.unipd.it/~rcardin/swea/2023/Diagrammi%20delle%20Classi.pdf>
 - *"Analisi e descrizione delle funzionalità^G: Diagrammi delle attività (UML)"*:
<https://www.math.unipd.it/~rcardin/swea/2022/Diagrammi%20di%20Attivit%C3%A0.pdf>
 - *"Progettazione^G: I pattern architetturali"*:
<https://www.math.unipd.it/~rcardin/swea/2022/Software%20Architecture%20Patterns.pdf>
 - *"Progettazione^G: Il pattern Dependency Injection"*:
<https://www.math.unipd.it/~rcardin/swea/2022/Design%20Pattern%20Architetturali%20-%20Dependency%20Injection.pdf>



- "Progettazione^G: Pattern Architeturali per le applicazioni con UI":
<https://www.math.unipd.it/~rcardin/sweb/2022/L02.pdf>
- "Progettazione^G: i pattern creazionali (GoF)":
<https://www.math.unipd.it/~rcardin/swea/2022/Design%20Pattern%20Creazionali.pdf>
- "Progettazione^G: I pattern strutturali (GoF)":
<https://www.math.unipd.it/~rcardin/swea/2022/Design%20Pattern%20Strutturali.pdf>
- "Progettazione^G: I pattern di comportamento (GoF)":
https://drive.google.com/file/d/1cpi6r0RMxFtC91nI6_sPrG1Xn-28z8eI/view?usp=sharing
- "Programmazione: SOLID programming":
<https://drive.google.com/file/d/1o1Xun2dVVc3mDiaGyN0FrDJhho031fLQ/view?pli=1>



2 Tecnologie

In questa sezione sono descritte le tecnologie utilizzate per lo sviluppo^G di **Second Brain**. Sono inclusi gli strumenti, i framework^G e le librerie utilizzati per lo sviluppo^G di backend, frontend e della parte di persistenza dei dati ma anche quelli di supporto, di verifica^G e i servizi esterni.

2.1 Tecnologie di codifica

Framework^G e librerie utilizzate per lo sviluppo^G backend e frontend.

2.1.1 TypeScript

TypeScript è un linguaggio basato su JavaScript che introduce la tipizzazione statica, migliorando la manutenibilità e la robustezza del codice. Tra le funzionalità^G offerte permette infatti di definire:

- Tipi Statici
- Interfacce
- Template (Generics)
- Enumerazioni

Il tutto è supportato dal compilatore statico *TSC* (*TypeScript Compiler*) che, inizialmente effettua la validazione^G del codice e dei tipi definiti senza l'esecuzione, e successivamente, in assenza di errori, produce codice JavaScript. Le direttive TypeScript sono disponibili solo in fase di sviluppo^G, poichè la "compilazione" termina comunque con la produzione di codice JavaScript.

Ciononostante, TypeScript ed il relativo compilatore sono uno strumento molto utile per verificare la correttezza del codice in fase di sviluppo^G (grazie anche ad un ricco supporto da parte degli IDE moderni) prima ancora della sua esecuzione, seguendo l'approccio "*fail fast*", ovvero l'obiettivo di trovare errori il prima possibile.

Il corretto utilizzo di TypeScript aiuta quindi a mitigare il debito tecnologico del progetto^G, motivo principale per il quale è stato inserito tra le tecnologie di sviluppo. Altra motivazione è la compatibilità con il framework^G Vue (vedere sezione successiva) utilizzato per lo sviluppo^G dell'interfaccia utente.

- **Versione:** 5.9.3
- **Documentazione:** <https://www.typescriptlang.org/docs/> (Consultato 09/04/2026)

2.1.2 Vue.js

Vue è un framework JavaScript open source per lo sviluppo di interfacce utente ed "SPA" (Single Page Application). Si basa su una logica definita per componenti, in cui ogni elemento dell'interfaccia utente viene implementato come componente singolo e poi integrato all'interno di altri componenti "*contenitori*" fino ad arrivare al componente "*radice*", andando a creare una gerarchia chiara e strutturata dell'intera interfaccia utente. In aggiunta ai componenti vengono definite anche le funzioni "*composables*", richiamabili all'interno dei componenti stessi. Le componenti possono essere sviluppate utilizzando JavaScript come linguaggio di scripting oppure, come effettuato nel nostro progetto^G, sfruttare il supporto a TypeScript per irrobustire l'architettura.

Punto di forza del framework^G è la possibilità di ottenere un'interfaccia utente particolarmente reattiva, grazie alla gestione dinamica dei dati tra i componenti basata sull'utilizzo di **props** ed **emits** (idiomi del framework^G): ogni componente infatti si aggiorna automaticamente in seguito a modifiche avvenute sui componenti collegati ad esso.



Vue inoltre mette a disposizione l'utilizzo del **router**, ulteriore strumento nativo del framework^G, con cui è possibile caricare dinamicamente contenuto all'interno della pagina senza eseguire ulteriori richieste: il documento HTML infatti viene richiesto solamente al primo accesso all'applicazione; una volta ottenuto il documento HTML "scheletro", il contenuto viene dinamicamente caricato e modificato attraverso Vue in base alla navigazione. Visivamente l'utente ha l'impressione di interagire con una "singola" pagina (da cui il termine SPA), in quanto il browser non aggiorna l'intero documento HTML della pagina ma sfrutta le funzionalità^G di Vue per aggiornare solamente le parti necessarie.

La scelta di utilizzare il framework^G Vue per il frontend rispetto a sviluppare unicamente utilizzando JavaScript puro è basata sulle seguenti motivazioni:

- **Organizzazione del codice:** ogni componente è costituito da 3 parti:
 - **template HTML:** contiene i tag HTML necessari al componente, muniti delle direttive di Vue (ex. `v-on` | `v-for` | `v-bind`).
 - **script TypeScript:** contiene la logica di comportamento compresa delle chiamate alle funzioni composables; viene sfruttato il supporto a TypeScript per ottenere componenti architetturealmente più robuste rispetto .
 - **stile CSS:** contiene il foglio di stile per il componente (applicato in maniera globale).

Tale suddivisione permette di avere una gestione più pulita del codice e di localizzare le eventuali modifiche, diminuendo enormemente il rischio di creare l'antipattern "*Big Ball of Mud*" (facilmente raggiungibile in caso si utilizzi solamente JavaScript puro).

- **Riutilizzo di codice:** ogni componente può essere riutilizzato in qualsiasi altro punto dell'interfaccia utente ed eventualmente personalizzato evitando la ripetizione di codice.
- **Manutenibilità del codice:** grazie alla logica "*a componenti*", ogni modifica o estensione di codice è localizzata in un punto unico, velocizzando e semplificando le attività di manutenzione.
- **Risorse disponibili:** la scelta del framework^G è stata basata per buona parte anche su tempo disponibile e capacità dei singoli membri del gruppo. Cofrontato con gli altri 2 principali framework^G per sviluppo^G frontend *Angular* e *React*, **Vue** è stato considerato come il giusto compromesso tra la rigidità di Angular e la flessibilità di React, in quanto offre le funzionalità^G necessarie al progetto^G senza una curva di apprendimento troppo alta che avrebbe impattato le risorse disponibili.
- **Versione:** 3.5+
- **Documentazione:** <https://vuejs.org/guide/introduction.html>

2.1.3 Marked.js

Marked.js è una libreria open-source scritta in JavaScript utilizzata per il parsing e la conversione di testo in formato Markdown. Consente di trasformare contenuti Markdown in HTML in modo rapido, supportando estensioni personalizzate e integrazione lato client e server.

- **Versione:**
- **Documentazione:** <https://marked.js.org/>

2.1.4 CSS

CSS (Cascading Style Sheets) è un linguaggio utilizzato per la formattazione di documenti HTML, XHTML e XML.

- **Versione:** CSS3
- **Documentazione:** <https://developer.mozilla.org/en-US/docs/Web/CSS>



2.1.5 Tailwind CSS

Tailwind CSS è un framework^G CSS open source. Offre una serie di classi CSS "di utilità". Queste classi vengono poi personalizzate e combinate secondo necessità per definire lo stile di ogni elemento.

- **Versione:** 4.2.2
- **Documentazione:** <https://tailwindcss.com/docs/installation/using-vite>

2.1.6 HTML

HTML (HyperText Markup Language) è il linguaggio di markup più utilizzato al mondo per i documenti web. È un linguaggio di pubblico dominio e le sue regole sono dettate da **W3C** (World Wide Web Consortium)

- **Versione:** HTML5
- **Documentazione:** <https://developer.mozilla.org/en-US/docs/Web/HTML>

2.1.7 PHP

PHP è un linguaggio di programmazione lato server progettato per lo sviluppo^G web, utilizzato per implementare la logica lato backend.

- **Versione:** 8.4.19
- **Documentazione:** <https://www.php.net/manual/it/>

2.1.8 Laravel

Laravel è un framework^G open source scritto in PHP utilizzato per lo sviluppo^G di applicazioni web moderne. Adotta il pattern architetturale MVC (Model View Controller) e fornisce strumenti integrati per la gestione del routing, dell'autenticazione e dell'accesso ai dati.

- **Versione:** 12.56.0
- **Documentazione:** <https://laravel.com/docs/13.x>

2.2 Tecnologie di verifica

Strumenti utilizzati per l'analisi del codice prodotto.

2.2.1 PHPUnit

PHPUnit è un framework^G utilizzato per l'analisi del codice PHP tramite test^G di unità. Lo scopo è trovare eventuali errori introdotti con modifiche al codice ed evitare che si verifichi una regressione.

- **Versione:** 11.5.55
- **Documentazione:** <https://phpunit.de/documentation.html>

2.3 Tecnologie per la persistenza dei dati (OPT)

2.3.1 PostgreSQL

PostgreSQL è un sistema di gestione di database relazionali open source compatibile con i sistemi operativi più diffusi.

- **Versione:** 17
- **Documentazione:** <https://www.postgresql.org/docs/>



2.4 Tecnologie di supporto

Strumenti utilizzati per facilitare lo sviluppo^G del prodotto da parte del gruppo.

2.4.1 Vite

Vite è uno strumento di sviluppo^G frontend che funge da dev server e build tool.

- **Versione:** 7.3.1
- **Documentazione:** <https://vite.dev/guide/>

2.4.2 Composer

Composer è un gestore di dipendenze per il linguaggio PHP utilizzato per installare, aggiornare e gestire automaticamente librerie e pacchetti necessari a un progetto.

- **Versione:** 2.9.5
- **Documentazione:** <https://getcomposer.org/doc/>

2.4.3 nginx

nginx è un web server open source leggero e ad alte prestazioni, utilizzabile anche come reverse proxy, load balancer, cache HTTP.

- **Versione:** 1.27
- **Documentazione:** <https://nginx.org/en/docs/>

2.4.4 PHP-FPM

- **Versione:** 8.4.19
- **Documentazione:** <https://www.php.net/manual/en/install.fpm.php>

2.4.5 Docker

Docker^G è una piattaforma di containerizzazione che consente di creare, distribuire ed eseguire applicazioni all'interno di ambienti isolati e leggeri, detti container. Ogni container^G include tutte le dipendenze necessarie al funzionamento del software ed è gestito in modo indipendente rispetto agli altri.

- **Versione:** 29.0.1
- **Documentazione:** <https://docs.docker.com/>

2.4.6 Docker Compose

Docker^G Compose è uno strumento che consente di definire, configurare e gestire applicazioni multi-container. Permette di gestire l'avvio, l'arresto e l'interazione tra più container^G in modo coordinato.

- **Versione:** 2.40.3
- **Documentazione:** <https://docs.docker.com/compose/>

2.5 Modelli LLM esterni

Tra i modelli LLM^G (Large Language Models) disponibili tramite l'endpoint Zucchetti, abbiamo selezionato il modello *Gemma3:4B*, in quanto ritenuto il più adeguato per le funzionalità^G AI richieste, sia in termini di velocità di risposta sia per la qualità e coerenza del contenuto generato.



3 Architettura

In questa sezione è descritta in modo dettagliato l'architettura del nostro prodotto. Verranno illustrate le nostre scelte progettuali e i componenti del nostro sistema.

3.1 Architettura logica

Il progetto^G Second Brain è strutturato in due componenti principali:

- **Backend:** responsabile^G della gestione della *logica di business* e della persistenza dei dati.
- **Frontend:** destinato alla presentazione dei dati ottenuti dal *backend*, il *frontend* consente all'utente di interagire con le funzionalità^G del progetto^G in modo intuitivo e user-friendly.

3.1.1 Backend

Nel *backend* si è scelto di utilizzare il framework^G Laravel, che permette la realizzazione di un servizio API^G REST. L'architettura del backend è suddivisa nei seguenti strati:

- **Presentation Layer:** si occupa del routing delle richieste; i controller fungono da intermediari tra il client e i servizi, gestendo il presentation layer e instradando le richieste del client ai servizi appropriati.
- **Application Layer:** gestisce la *logica di business* dell'applicazione; questo strato è responsabile^G delle operazioni e delle regole di business, garantendo che tutte le operazioni siano eseguite correttamente;
- **Data Access Layer:** strato che gestisce la persistenza dei dati, fornendo i mezzi per salvare, recuperare e manipolare i dati.

La suddivisione in questi strati facilita lo sviluppo^G, la manutenzione e la scalabilità del *backend* del progetto^G Second Brain, conformandosi all'architettura **multi-tier** a 3-tier.

3.1.1.1 Gestione note e Esportazione

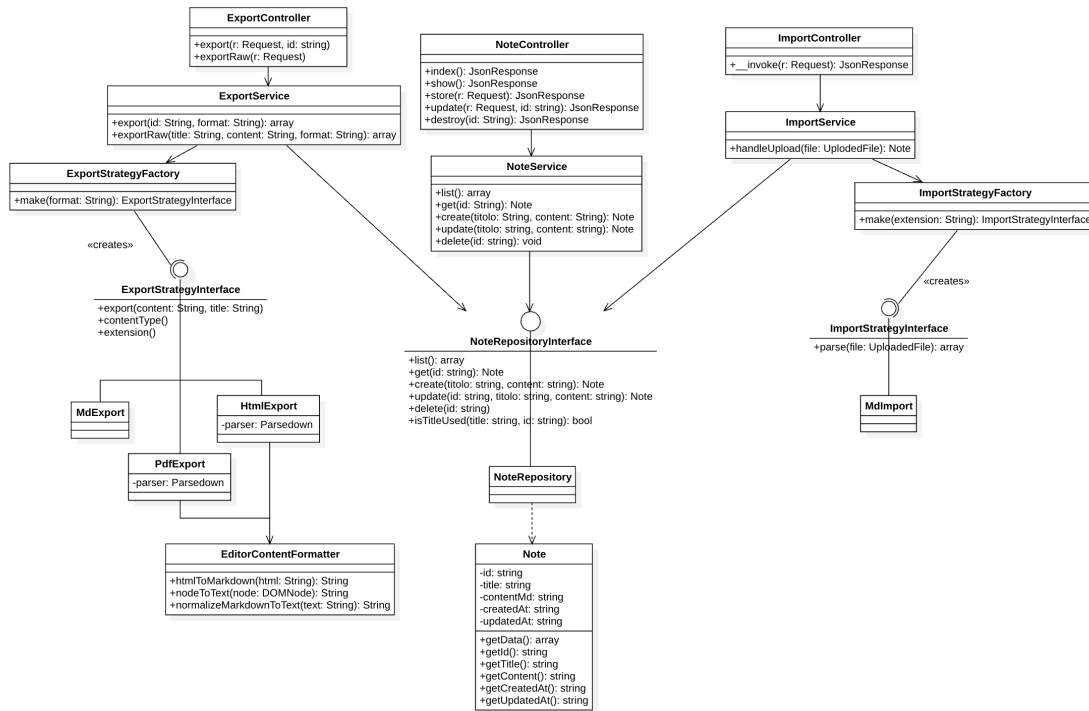


Figura 1: Diagramma delle classi per Gestione note - Esportazione - Importazione

3.1.1.1.1 NoteController È il controller HTTP responsabile^G della gestione delle richieste REST^G relative alle note. Appartiene al layer di presentazione dell'architettura layered e funge da punto di ingresso per tutte le operazioni CRUD espone dall'API. Si limita a validare i dati in ingresso, delegare l'elaborazione a *NoteService* e restituire una risposta HTTP appropriata. Segue la descrizione dei metodi:

- **index()** : **JsonResponse** - gestisce le richieste GET `/notes`, restituendo la lista completa delle note in formato JSON con status 200, delegando il recupero dei dati a *NoteService*
- **show(string \$id)** : **JsonResponse** - gestisce le richieste GET `/notes/{id}` e restituisce i dettagli di una singola nota identificata dall'id fornito
- **store(Request \$request)** : **JsonResponse** - gestisce le richieste POST `/notes`, valida i campi, delega a *NoteService* la creazione e restituisce la nota creata con status 201
- **update(Request \$request, string \$id)** : **JsonResponse** - gestisce le richieste PUT `/notes/{id}`, valida i campi e delega l'aggiornamento a *NoteService*, passando l'id e i nuovi dati
- **destroy(string \$id)** : **JsonResponse** - gestisce le richieste DELETE `/notes/{id}`, delega l'eliminazione a *NoteService* e restituisce una risposta vuota con status 204 in caso di successo

3.1.1.1.2 NoteService È il service responsabile^G della logica di business relativa alle note. Appartiene al layer applicativo dell'architettura layered e funge da intermediario tra il controller e il repository. Dipende esclusivamente dall'interfaccia *NoteRepositoryInterface*, garantendo il rispetto del principio di inversione delle dipendenze. Segue la descrizione dei metodi:

- **list()** : **array** - restituisce la lista completa delle note, delegando direttamente al repository^G
- **get(string \$id)** : **array** - restituisce i dettagli di una singola nota tramite il suo id



- `create(string $title, string $contentMd) : array` - gestisce la creazione di una nuova nota, verificando che il titolo non sia già in uso prima di delegare al repository^G
- `update(string $id, string $title, string $contentMd) : array` - gestisce l'aggiornamento di una nota esistente; prima di delegare al repository^G verifica^G che il titolo non sia già in uso, escludendo dal confronto la nota corrente così che una nota possa essere salvata con il proprio titolo invariato senza incorrere in un falso positivo
- `delete(string $id) : void` - delega l'eliminazione della nota al repository^G

3.1.1.1.3 NoteRepositoryInterface Interfaccia che definisce il contratto che ogni implementazione del repository^G di note deve rispettare. Appartiene al layer di accesso ai dati dell'architettura layered. Dichiarando la dipendenza su questa interfaccia, `NoteService` può funzionare con qualsiasi implementazione senza richiedere modifiche al codice applicativo. Segue la descrizione dei metodi:

- `list() : array` - restituisce la lista completa delle note disponibili, ogni elemento dell'array contiene i metadati della nota senza il contenuto completo
- `get(string $id) : Note` - restituisce i dati completi di una singola nota identificata dall'id fornito
- `create(string $title, string $contentMd) : Note` - crea una nuova nota con il titolo e il contenuto Markdown forniti, genera un identificatore e registra il timestamp di creazione
- `update(string $id, string $title, string $contentMd) : Note` - aggiorna il titolo e il contenuto di una nota esistente assieme al timestamp di ultima modifica
- `delete(string $id) : void` - elimina definitivamente la nota identificata dall'id fornito
- `isTitleUsed(string $title, ?string $excludeId = null) : bool` - verifica^G se un titolo è già utilizzato da un'altra nota; `excludeId` permette di escludere dal confronto una nota specifica, per evitare che tale nota venga bloccata dal confronto con se stessa

3.1.1.1.4 NoteRepository È l'implementazione concreta di `NoteRepositoryInterface` che persiste le note come file Markdown sul filesystem locale tramite Laravel Storage. Realizza tutti i metodi dichiarati nell'interfaccia: `list()`, `get()`, `create()`, `update()`, `delete()`, `isTitleUsed()`; per la descrizione della firma e del contratto di ciascun metodo si rimanda alla sezione dedicata all'interfaccia.

3.1.1.1.5 Note È una classe Model che rappresenta l'entità Nota nel dominio dell'applicazione. Esistono due tipi di metodo:

- `getData() : array` - serializza l'oggetto in array, utile per passare i dati ai layer superiori
- getter singoli (`getId()`, `getTitle()`, ...) utili quando serve accedere ad una singola proprietà senza serializzare tutto

3.1.1.1.6 ImportController È il controller HTTP responsabile^G della gestione delle richieste di importazione. Appartiene al layer di presentazione dell'architettura layered ed è implementato come controller invocabile dato che la sua responsabilità è circoscritta ad una sola operazione. Si limita a validare il file, delegare l'importazione a *ImportService* e costruire la risposta HTTP. Segue la descrizione dei metodi:

- `__invoke(Request $request) : JsonResponse` - valida il parametro (cioè il campo file della richiesta), delega l'importazione a *ImportService*, passando il file caricato; in caso di successo restituisce la nota importata in formato JSON con status 201, altrimenti ritorna una risposta JSON con status 400.



3.1.1.1.7 ImportService È il service responsabile^G della logica di importazione di una nota. Appartiene al layer applicativo dell'architettura layered. Dipende da `NoteRepositoryInterface` garantendo il disaccoppiamento dall'implementazione del layer di persistenza. Segue la descrizione dei metodi:

- `handleUpload(UploadedFile $file) : Note` - estrae l'estensione del file caricato, delega la creazione della strategy corretta a `ImportStrategyFactory` ed invoca sulla strategy il metodo per estrarre il contenuto in Markdown; successivamente aggiunge un timestamp al titolo della nota per evitare conflitti e invoca `NoteRepositoryInterface::create()`, ritornando un array.

3.1.1.1.8 ImportStrategyFactory - È la factory responsabile^G della creazione della strategy di importazione corretta in base all'estensione del file caricato. Appartiene al layer applicativo dell'architettura layered e implementa il pattern Factory Method, centralizzando la logica di selezione della strategy. Segue la descrizione dei metodi:

- `make(string $extension) : ImportStrategyInterface` - metodo statico che istanzia la strategy concreta corrispondente all'estensione del file caricato.

3.1.1.1.9 ImportStrategyInterface Interfaccia che definisce il contratto che ogni strategy di importazione deve rispettare. Appartiene al layer applicativo dell'architettura layered e rappresenta il punto di astrazione del pattern strategy applicato all'importazione delle note. Segue la descrizione dei metodi:

- `parse(UploadedFile $file) : array` - estrae il contenuto in Markdown e il titolo della note dal file caricato.

3.1.1.1.10 MdImport È l'implementazione concreta di `ImportStrategyInterface` per l'importazione di file `.md`. Per la descrizione della firma e del contratto di ciascun metodo si rimanda alla sezione dedicata all'interfaccia.

3.1.1.1.11 ExportController È il controller HTTP responsabile^G della gestione delle richieste di esportazione, rendendo disponibile una nota come file scaricabile. Appartiene al layer di presentazione dell'architettura layered ed è implementato come controller invocabile dato che la sua responsabilità è circoscritta ad una sola operazione. Si limita a validare il formato richiesto, delega l'esportazione a `ExportService` e costruisce la risposta HTTP con gli header appropriati per il download dei file. Segue la descrizione dei metodi:

- `export(Request $request, string $id)` - valida il parametro (cioè il formato con cui si vuole esportare la nota) e delega l'esportazione a `ExportService`, passando l'id della nota e il formato richiesto; in caso di successo restituisce una risposta binaria in modo che il browser tratti la risposta come un file da scaricare, altrimenti ritorna una risposta JSON con status 404
- `exportRaw(Request $request)` - valida i parametri (cioè il titolo, il contenuto e il formato con cui si vuole esportare la nota) e delega l'esportazione a `ExportService`, passando i diversi parametri e il formato richiesto; in caso di successo restituisce una risposta binaria in modo che il browser tratti la risposta come un file da scaricare, altrimenti ritorna una risposta JSON con status 404

3.1.1.1.12 ExportService È il service responsabile^G della logica di esportazione di una nota in un formato file. Appartiene al layer applicativo dell'architettura layered e gestisce la collaborazione tra il repository^G e la factory delle strategy di esportazione. Dipende da `NoteRepositoryInterface`, garantendo il disaccoppiamento dal layer di persistenza. Segue la descrizione dei metodi:

- `export(string $id, string $format) : array` - recupera la nota identificata dall'id tramite il repository^G, delega la creazione della strategy corretta a `ExportStrategyFactory` in base al formato richiesto ed invoca sulla strategy i diversi metodi per ottenere il contenuto del file esportato, il MIME type e l'estensione



- `exportRaw(string $title, string $content, string $format) : array` - delega la creazione della strategy corretta a *ExportStrategyFactory* in base al formato richiesto ed invoca sulla strategy i diversi metodi per ottenere il contenuto del file esportato, il MIME type e l'estensione

3.1.1.1.13 ExportStrategyFactory È la factory responsabile^G della creazione della strategy di esportazione corretta in base al formato richiesto. Appartiene al layer applicativo dell'architettura layered e implementa il pattern Factory Method, centralizzando la logica di selezione della strategy. Segue la descrizione dei metodi:

- `make(string $format) : ExportStrategyInterface` - metodo statico che istanzia e restituisce la strategy concreta corrispondente al formato richiesto

3.1.1.1.14 ExportStrategyInterface Interfaccia che definisce il contratto che ogni strategy di esportazione deve rispettare. Appartiene al layer applicativo dell'architettura layered e rappresenta il punto di astrazione del pattern strategy applicato all'esportazione delle note. Segue la descrizione dei metodi:

- `export (string $content, string $title) : string` - trasforma il contenuto Markdown della nota nel formato di output specifico della strategy
- `contentType() : string` - restituisce il MIME type associato al formato di esportazione, per consentire al browser di interpretare correttamente il file ricevuto
- `extension() : string` - restituisce l'estensione del file associata al formato di esportazione, utilizzata per costruire il nome del file

3.1.1.1.15 MdExport È l'implementazione concreta di *ExportStrategyInterface* per l'esportazione nel formato Markdown, è la strategy più semplice del sistema in quanto il contenuto delle note è già nativamente in formato Markdown, quindi non richiede alcuna trasformazione. Per la descrizione della firma e del contratto di ciascun metodo si rimanda alla sezione dedicata all'interfaccia.

3.1.1.1.16 HtmlExport È l'implementazione concreta di *ExportStrategyInterface* per l'esportazione nel formato HTML. Utilizza la libreria *Parsedown* per convertire il contenuto Markdown della nota in markup HTML, che viene poi incorporato in un documento HTML completo e ben formato. Per la descrizione della firma e del contratto di ciascun metodo si rimanda alla sezione dedicata all'interfaccia.

3.1.1.1.17 PdfExport È l'implementazione concreta di *ExportStrategyInterface* per l'esportazione nel formato PDF. Prima delega al parser la trasformazione del Markdown in HTML, poi utilizza la libreria *barryvdh/laravel-dompdf* per convertire l'HTML in un documento PDF binario. Per la descrizione della firma e del contratto di ciascun metodo si rimanda alla sezione dedicata all'interfaccia.

3.1.1.1.18 EditorContentFormatter Utility dedicata alla conversione di contenuti HTML generati da editor testuali in testo normalizzato in formato Markdown, ha l'obiettivo di eliminare elementi nascosti o metadati che utilizziamo nell'editor per la visualizzazione di blocchi in seguito alla generazione di testo con LLM. Segue la descrizione dei metodi:

- `htmlToMarkdown(string $html) : string` - riceve in input una stringa contenente HTML e restituisce una versione testuale normalizzata del contenuto
- `nodeToText(DOMNode $node) : string` - metodo ricorsivo utilizzato per convertire i nodi del DOM in contenuto testuale
- `normalizeMarkdownText(string $text) : string` - metodo di normalizzazione finale del testo estratto

3.1.1.2 Interazione con LLM^G

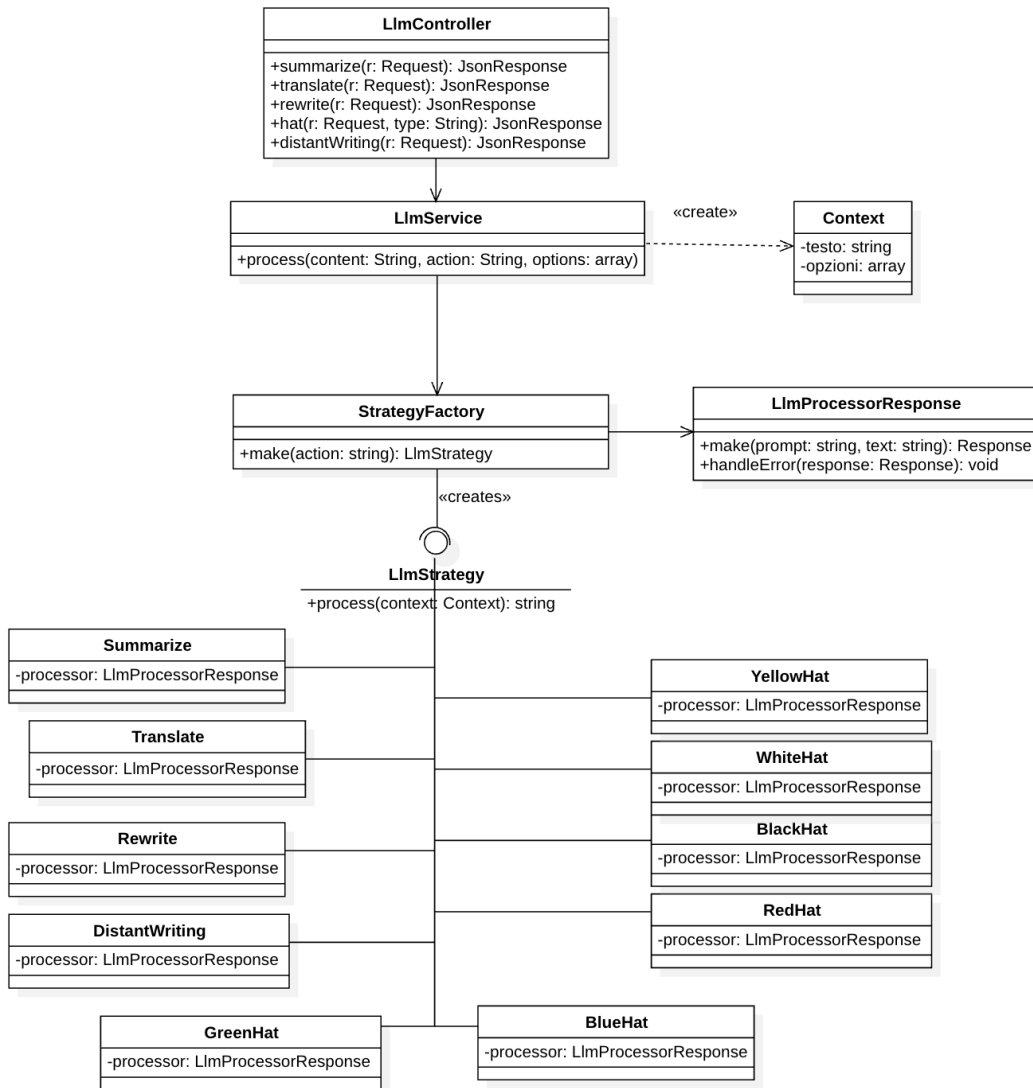


Figura 2: Diagramma delle classi per Interazione con LLM

3.1.1.2.1 LlmController È il controller HTTP responsabile^G della gestione delle richieste di elaborazione del testo tramite modelli linguistici. Appartiene al layer di presentazione dell'architettura layered. Si limita a validare i dati in ingresso, delegare l'elaborazione a *LlmService* e restituire il risultato in formato JSON. Segue la descrizione dei metodi:

- `summarize(Request $request) : JsonResponse` - valida i parametri (il content), delega l'elaborazione a *LlmService* e restituisce il risultato in una risposta JSON con status 200 in caso di successo, mentre con status 502 in caso di errore.
- `translate(Request $request) : JsonResponse` - valida i parametri (il content e la lingua della traduzione), delega l'elaborazione a *LlmService* e restituisce il risultato in una risposta JSON con status 200 in caso di successo, mentre con status 502 in caso di errore.

- `rewrite(Request $request) : JsonResponse` - valida i parametri (il content e il tono della riscrittura), delega l'elaborazione a *LlmService* e restituisce il risultato in una risposta JSON con status 200 in caso di successo, mentre con status 502 in caso di errore.
- `hat(Request $request, string $type) : JsonResponse` - valida i parametri (il content), delega l'elaborazione a *LlmService* e restituisce il risultato in una risposta JSON con status 200 in caso di successo, mentre con status 502 in caso di errore; nel caso in cui il tipo di cappello passato come parametro non sia supportato restituisce una risposta con status 400.
- `distantWriting(Request $request) : JsonResponse` - valida i parametri (il content), delega l'elaborazione a *LlmService* e restituisce il risultato in una risposta JSON con status 200 in caso di successo, mentre con status 502 in caso di errore.

3.1.1.2.2 LlmService È il service responsabile^G della logica di elaborazione del testo tramite modelli linguistici. Appartiene al layer applicativo dell'architettura layered e gestisce la collaborazione tra *LlmStrategyFactory* e le strategy di elaborazione, che ne eseguono il processo. Si limita a costruire il contesto di esecuzione e delegare il lavoro alle classi di servizio. Segue la descrizione dei metodi:

- `process(string $content, string $action, array $options = []) : string` - costruisce un oggetto *LlmContext* con il testo da elaborare e le opzioni aggiuntive, delega la selezione e creazione della strategy a *LlmStrategyFactory* in base all'azione e esegue la strategy. Infine restituisce il testo prodotto dalla strategy.

3.1.1.2.3 LlmStrategyFactory È la factory responsabile^G della creazione della strategy di elaborazione testuale, elaborata in base all'azione richiesta. Appartiene al layer applicativo dell'architettura layered e implementa il pattern Factory Method, centralizzando la logica di selezione della strategy. Segue la descrizione dei metodi:

- `make(string $action) : LlmStrategyInterface` - metodo statico che istanzia e restituisce la strategy concreta corrispondente all'azione richiesta.

3.1.1.2.4 LlmProcessorResponse Classe utility che gestisce le chiamate a un'API^G LLM^G tramite il client HTTP di Laravel. Segue la descrizione dei metodi:

- `make(string $prompt, string $text) : Response` - costruisce e invia una richiesta POST all'endpoint dell'API^G configurata, restituisce la Response grezza di Laravel
- `handleError(Response $response) : void` - valida la risposta e lancia una eccezione in caso di errore, se la risposta è andata a buon fine non fa nulla

3.1.1.2.5 LlmStrategyInterface Interfaccia che definisce il contratto che ogni strategy di elaborazione testuale tramite modelli linguistici deve rispettare. Appartiene al layer applicativo dell'architettura layered e rappresenta il punto di astrazione del pattern strategy applicato alla elaborazione del testo. Segue la descrizione dei metodi:

- `process(LlmContext $context) : string` - riceve un oggetto *LlmContext* contenente il testo da elaborare e le opzioni aggiuntive, esegue la chiamata al modello linguistico con il prompt specifico della strategy e restituisce il testo prodotto come string.

3.1.1.2.6 LlmContext È un oggetto di contesto che incapsula i dati necessari all'esecuzione di una strategy di elaborazione testuale; funge da contenitore di informazioni che vengono passata dalla *LlmService* alla strategy selezionata, evitando il passaggio di parametri multipli. Si limita a esporre i dati tramite getter.



3.1.1.2.7 Strategy concrete Tutte le strategy implementano *LlmStrategyInterface* e condividono la stessa struttura interna: costruiscono una chiamata all'API^G del modello linguistico con un prompt specifico per il tipo di richiesta di elaborazione tramite *LlmProcessorResponse* e restituiscono il contenuto generato dal modello.

- **Summarize** - elabora il testo producendo un riassunto che mantiene solo le informazioni più rilevanti, preservando la voce narrativa originale
- **Translate** - traduce il testo nella lingua specificata tramite il campo dell'array *options* del contesto
- **Rewrite** - riscrive il testo in base alle opzioni scelte dall'utente (variazione stilistica, estensione del testo, correzione grammaticale o miglioramento del lessico)
- **DistantWriting** - genera testo originale a partire da una descrizione fornita dall'utente
- **WhiteHat** - analizza il testo da un punto di vista fattuale, identificando dati oggettivi, informazioni verificabili e lacune conoscitive presenti nel contenuto
- **RedHat** - analizza il testo esprimendo le reazioni emotive e le impressioni intuitive che suscita
- **BlackHat** - individua rischi, criticità, punti deboli e potenziali conseguenze negative presenti nel testo
- **YellowHat** - identifica i punti di forza, benefici e aspetti positivi del testo
- **GreenHat** - propone idee alternative e riformulazioni possibili del contenuto
- **BlueHat** - valuta la chiarezza espositiva e il processo comunicativo del testo, fornendo una riflessione sull'organizzazione del contenuto



3.1.2 Frontend

Il *frontend* dell'applicazione è sviluppato con il framework^G Vue.js, adottando nativamente il pattern architetturale *MVVM* (Model-View-ViewModel), che separa le responsabilità tra logica di business, stato dell'interfaccia e presentazione visiva. In particolare, il ViewModel è implementato tramite la *Composition API*^G, che gestisce lo stato locale del componente, reagisce ai cambiamenti del Model e aggiorna la View in modo reattivo. La struttura delle cartelle è la seguente:

```
ts/
├── components/
│   ├── # Componenti UI base, comment
│   ├── icons/
│   │   └── # Componenti icone, comment
│   ├── layout/
│   │   ├── # Componenti di layout, comment
│   │   └── navigation/
│   │       └── # Bottoni di navigazione (NavButton, NavItem), comment
├── composables/
│   └── # Composables dell'applicazione, comment
├── errors/
│   └── # Errori custom, comment
├── layout/
│   └── AppLayout.vue
├── pages/
│   ├── NoteEditorPage.vue
│   └── NoteArchivePage.vue
├── router/
│   └── index.ts
├── services/
│   └── # File per chiamate APIG al backend attraverso la libreria Axios, comment
├── tests/
│   └── # TestG unitari del frontend, comment
├── types/
│   └── # Tipi utilizzati globalmente nell'applicazione, comment
├── utils/
│   └── # Funzioni di utilità, comment
├── app.ts
└── App.vue
```

3.1.2.1 Root Component

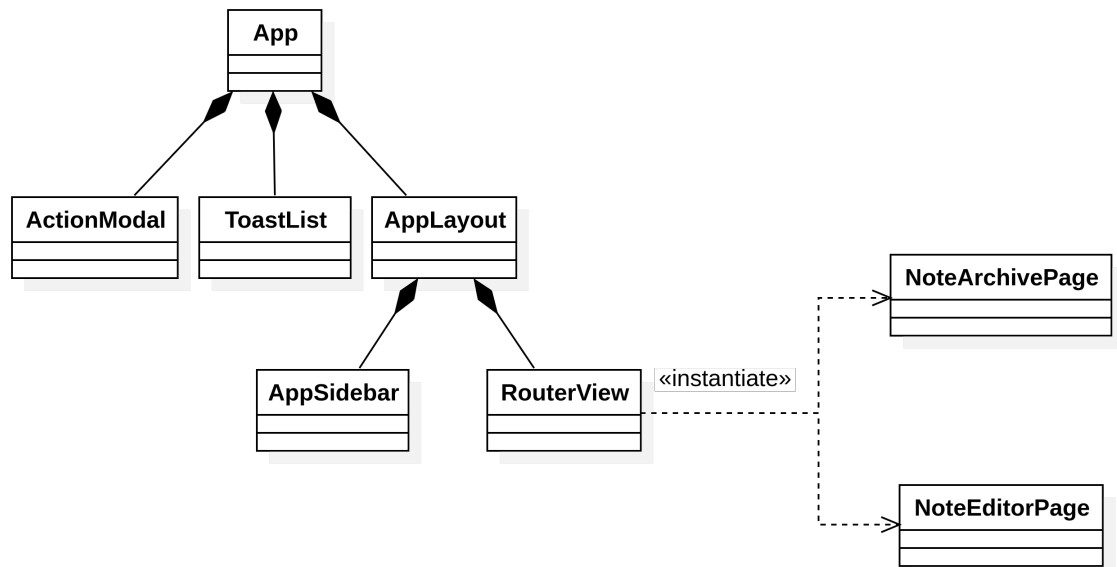


Figura 3: Diagramma UML gerarchia principale frontend

3.1.2.1.1 App Rappresenta il componente radice dell'applicazione, responsabile^G per l'inizializzazione dell'interfaccia. Al suo interno vengono istanziati i componenti trasversali all'intera applicazione: **ActionModal**, **ToastList** e **AppLayout**.

3.1.2.1.2 AppLayout Componente che rappresenta il layout principale dell'applicazione. Al suo interno vengono definiti la barra laterale di navigazione (**AppSidebar**) e l'area di contenuto dinamico (**RouterView**). Il componente garantisce che la struttura della pagina rimanga stabile mentre il contenuto centrale viene sostituito in base al URL attivo.

3.1.2.1.3 RouterView Componente nativo di **Vue Router** che funge da contenitore dinamico all'interno di **AppLayout**, responsabile^G del rendering della pagina corrispondente alla rotta URL attiva. A seconda del percorso di navigazione, il componente monta in tempo reale la pagina appropriata: **NoteEditorPage** (pagina dell'editor della nota) e **NoteArchivePage** (pagina dell'archivio delle note). La mappatura è definita in **index.ts**:

- / - la radice è configurata per reindirizzare verso l'archivio delle note, rendendo la pagina la vista iniziale dell'applicazione
- /notes/new - route che reindirizza alla pagina di editor vuota, per la creazione di una nuova nota
- /notes/:id - route che permette di aprire una nota e visualizzarla nella pagina di editor, dove :id rappresenta un parametro dinamico che identifica univocamente la nota da caricare

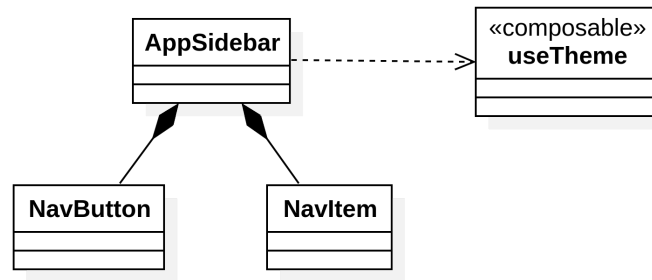


Figura 4: Diagramma UML componente AppSidebar

3.1.2.1.4 AppSidebar Componente di navigazione principale dell'applicazione, reso come elemento `<aside>` persistente sul lato sinistro dell'interfaccia. Implementa un comportamento di apertura / chiusura reattivo: tramite **Vue Router** si chiude automaticamente quando l'utente apre l'editor dell'applicazione. L'utente può controllare la visibilità manualmente tramite un pulsante di toggle. All'interno sono definiti due componenti, **NavButton** e **NavItem**, e un selettore per il tema dell'applicazione.

3.1.2.1.5 NavButton Bottone che rappresenta una chiamata all'azione - segnala all'utente un'operazione da compiere. Segue la descrizione dei **prop**:

- **string \$label** - label del bottone
- **string \$to** - rotta di reindirizzamento

3.1.2.1.6 NavItem Bottone che rappresenta una destinazione di navigazione - segnala all'utente dove si trova nell'applicazione (la route attiva). Quando il link corrisponde alla rotta corrente, viene applicato uno stile visivo distinto, che funge da indicatore di posizione nella navigazione. Segue la descrizione dei **prop**:

- **string \$label** - label del bottone
- **string \$to** - rotta di reindirizzamento

3.1.2.1.7 useTheme Composable che si occupa della gestione del tema visivo dell'applicazione (chiaro / scuro / sistema); recupera la preferenza salvata in **localStorage** e lo applica in modo immediato, garantendo una visione persistente tra sessioni diverse. In assenza di una preferenza salvata, il tema predefinito è **system**, che delega la scelta al sistema operativo. Segue la descrizione dei metodi esposti:

- **setTheme(Theme \$t)** - applica il tema passato dalla funzione e lo memorizza nella variabile **theme**.

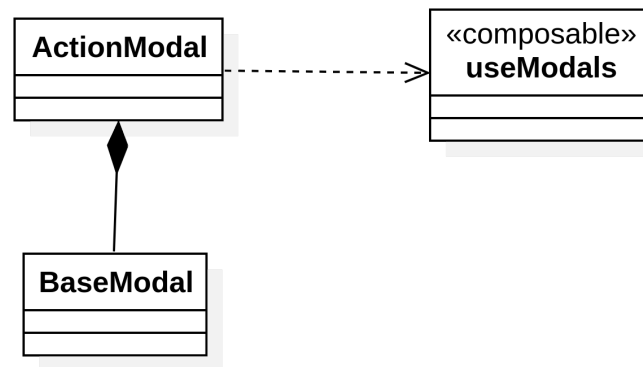


Figura 5: Diagramma UML componente ActionModal

3.1.2.1.8 ActionModal Componente che centralizza tutte le finestre di dialogo dell'applicazione, fungendo da registro globale dei modal. Tutti i modali vengono dichiarati in un unico punto e resi disponibili globalmente tramite il composabile `useModals`. Ogni modale espone tramite *scoped slot* le variabili `resolve` (comunica il valore di chiusura) e `args` (argomenti passati all'apertura). I modali strutturati sono:

- **RenamePromise** - modale per la rinomina, composto da un input testuale per inserire il nome desiderato
- **SaveAsPromise** - modale per il salvataggio con nome, simile al modale della rinomina
- **DeletePromise** - modale semplice per l'eliminazione, con un bottone di conferma
- **ClonePromise** - modale per la clonazione, composto da un input testuale per inserire il nome della nota clonata. Di default presenta un nome univoco all'archivio per facilitare l'azione all'utente
- **SavePromise** - modale per il salvataggio con conflitto, ovvero quando si presenta un incongruenza tra le modifiche locali e la versione persistita nell'archivio. Mostra tre opzioni: salvataggio con nome, aggiornamento della nota esistente o sovrascrittura della nota presente in archivio
- **DiscardPromise** - modale di avviso per il salvataggio della nota, azionato quando l'utente vuole uscire dall'editor della nota con modifiche non salvate

3.1.2.1.9 BaseModal Componente che definisce la struttura scheletrica comune a tutti i dialog dell'applicazione, esponendo tre slot (header, body e footer) che permettono ai componenti di iniettare il contenuto specifico mantenendo una presentazione visiva coerente. Segue la descrizione dei `prop`:

- `string $title` - titolo del modale

3.1.2.1.10 useModals Composabile che si occupa della gestione dei dialog dell'applicazione, implementato tramite il pattern *Promise-based modal*, offerto dalla funzione `createTemplatePromise` della libreria `VueUse`. Ogni modale è modellato come una `Promise`: quando viene aperto, restituisce una `Promise` tipizzata che si risolve con il valore corrispondente alla scelta dell'utente, permettendo ai componenti di attenderne il risultato in modo asincrono tramite `await`. Tutti i modali sono configurati come *Singleton*.

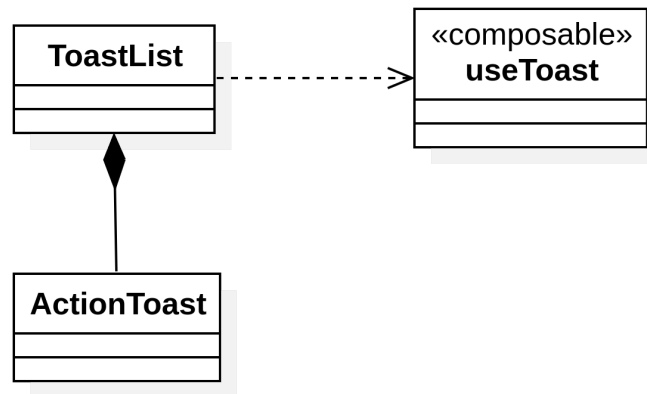


Figura 6: Diagramma UML componente ToastList

3.1.2.1.11 ToastList Componente responsabile^G del rendering delle notifiche toast attiva nell'applicazione. Il componente utilizza il composable useToast per utilizzare la lista delle notifiche dinamica, da cui ottiene l'elenco dei toast attivi. Per ogni elemento della lista il componente renderizza un **ActionToast**, passandogli i dati necessari e delegando la gestione della chiusura tramite l'evento `@close`.

3.1.2.1.12 ActionToast Componente che definisce la struttura di un singolo toast, composto dal titolo, il messaggio e il pulsante di chiusura. Il prop `type` determina la variante della notifica, che può essere di successo, di errore, di avvertenza o di info. Tramite una serie di mappe il `type` è associato ad una combinazione distinta di colori, che identificano visualmente il tipo del toast. L'icona nello stesso modo viene selezionata dinamicamente. Segue la descrizione dei `prop`:

- `string $title` - titolo del toast
- `string $message` - messaggio della notifica
- `ToastType $type` - tipo del toast. `ToastType` è un *Union Type* definito nel composable useToast

Segue la descrizione degli `emit`:

- `close: []` - evento emesso al click del pulsante di chiusura

3.1.2.1.13 useToast Composable che si occupa della gestione dello stato globale delle notifiche toast. Mantiene una lista reattiva `toasts` di notifiche attive. Ogni elemento della lista è identificato univocamente da un id generato tramite `Date.now()`. L'aggiunta dei toast è gestita tramite la funzione interna `updateState`, che pianifica la rimozione automatica tramite un timeout, definito di default come `defaultTimeout` o `defaultErrorTimeout` (di lunghezza maggiore). Segue la descrizione dei metodi esposti:

- `successToast(string $title, string $message, number $timeout = defaultTimeout)` - chiama `updateState` con il type success
- `errorToast(string $title, string $message, number $timeout = defaultErrorTimeout)` - chiama `updateState` con il type error
- `warningToast(string $title, string $message, number $timeout = defaultTimeout)` - chiama `updateState` con il type warning
- `infoToast(string $title, string $message, number $timeout = defaultTimeout)` - chiama `updateState` con il type info
- `removeToast(number $id)` - rimuove nella lista di toast quello corrispondente all'id passato nella funzione

3.1.2.2 Comunicazione con backend

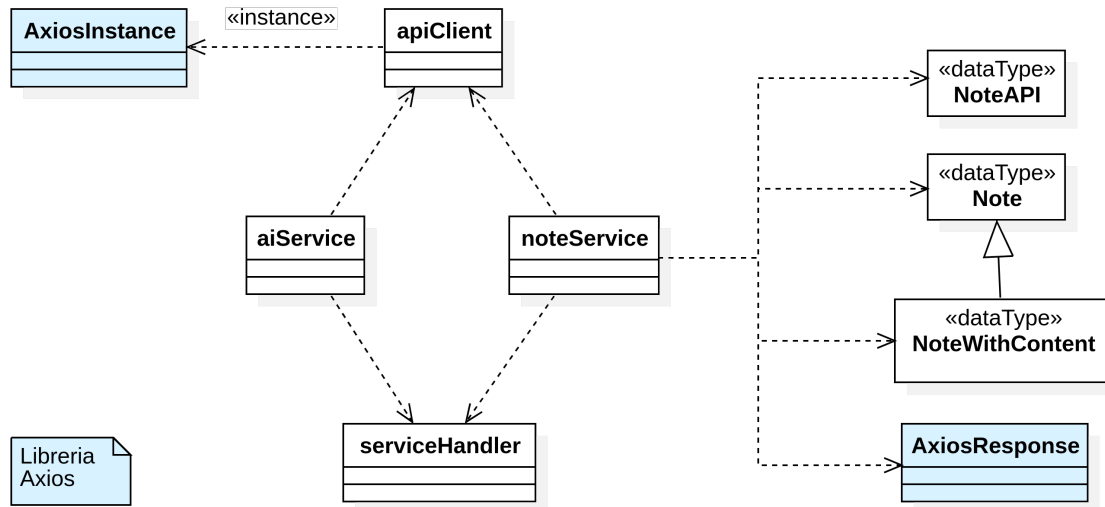


Figura 7: Diagramma UML struttura comunicazione con backend

3.1.2.2.1 Note (type) Modello base contenente i metadati della nota:

- string \$id - id univoco della nota
- string \$name - nome della nota
- string \$last_edit - data dell'ultima modifica
- string \$creation - data della creazione

3.1.2.2.2 NoteWithContent (type) Estensione di Note:

- string \$content - contenuto della nota

3.1.2.2.3 NoteAPI (type) Modello della nota restituito dal backend:

- string \$id - id univoco della nota
- string \$title - nome della nota
- string \$updated_at - data dell'ultima modifica
- string \$created_at - data della creazione
- string \$content_md - contenuto della nota

3.1.2.2.4 apiClient Definisce una istanza centralizzata di *Axios* (*AxiosInstance*) per comunicare con il backend. Configura i parametri **baseUrl**, **timeout**, headers HTTP e la gestione delle risposte. Include inoltre un interceptor per gestire gli errori di export convertendo le risposte da *Blob* in oggetti *JSON* leggibili dal frontend.



3.1.2.2.5 noteService Modulo che centralizza la gestione delle operazioni CRUD e delle funzionalità^G di import / export delle note tramite API^G REST^G, utilizzando `apiClient` per la comunicazione con il backend. Include funzioni di formattazione per la data (`formatDate`) e per la nota ricevuta (da `NoteAPI` a `Note` / `NoteWithContent`). Segue la descrizione dei metodi esposti:

- `getAll(): Promise<Note[]>` - ottiene la lista intera delle note archiviate nel server locale
- `get(string $id): Promise<NoteWithContent>` - riceve l'intera nota (metadati e contenuto) in base all'id univoco
- `rename(string $id, string $newName): Promise<Note>` - aggiorna il nome della nota con identificativo `id`
- `remove(string $id): Promise<void>` - elimina una nota con identificativo `id`
- `store(string $name, string $content): Promise<Note>` - archivia una nuova nota nel server locale e ritorna i metadati della nota generati (`id`, `created_at`, `content_md`)
- `update(string $id, string $title, string $content): Promise<NoteWithContent>` - aggiorna il nome e il contenuto della nota con identificativo `id`
- `export(string $id, 'pdf' | 'md' | 'html' $format): Promise<void>` - esporta la nota con identificativo `id` nel formato specificato
- `exportRaw(string $name, string $content, 'pdf' | 'md' | 'html' $format): Promise<void>` - utilizza il backend per esportare una nota non presente nell'archivio nel formato specificato
- `import(FormData $file): Promise<Note>` - importa una nota in formato `md` e ritorna la nota insieme ai metadati

3.1.2.2.6 aiService Modulo che centralizza le richieste API^G relative all'interazione con i servizi di IA, utilizzando `apiClient` per la comunicazione con il backend. Il modulo espone le seguenti interfacce:

- `AiAction` - *Union Type* che indica il tipo di azione: `summarize`, `translate`, `rewrite`, `blackhat`, `bluehat`, `greenhat`, `redhat`, `whitehat`, `yellowhat`, `distant writing`
- `AiTone` - *Union Type* che indica il tono della riscrittura: `grammar`, `extension`, `lexicon`, `stylistic`
- `AiLang` - *Union Type* che indica la lingua della traduzione: `it`, `en`, `fr`, `de`, `es`, `pt`
- `AiOptions` - interfaccia che ha due proprietà: `style`, una tupla di `AiTone` (minimo 1 valore), e la lingua opzionale `lang`

Segue la descrizione dei metodi esposti:

- `process(string $content, AiAction $action, AiOptions $options): Promise<string>` - compone l'endpoint API^G corretto in base all'azione ed invia una richiesta POST con `apiClient`
- `translate(string $content, AiLang $lang = 'en'): Promise<string>` - traduce il contenuto nella lingua desiderata
- `summarize(string $content): Promise<string>` - riassume il contenuto
- `rewrite(string $content, [AiTone, ...AiTone[]] $style): Promise<string>` - riscrive il contenuto in base ai toni desiderati
- `blackhat(string $content): Promise<string>` - critica il contenuto secondo il cappello nero

- `bluehat(string $content): Promise<string>` - critica il contenuto secondo il cappello blu
- `greenhat(string $content): Promise<string>` - critica il contenuto secondo il cappello verde
- `redhat(string $content): Promise<string>` - critica il contenuto secondo il cappello rosso
- `whitehat(string $content): Promise<string>` - critica il contenuto secondo il cappello bianco
- `yellowhat(string $content): Promise<string>` - critica il contenuto secondo il cappello giallo
- `distantWriting(string $content): Promise<string>` - ritorna una risposta in base al prompt inviato tramite content

3.1.2.2.7 serviceHandler Funzione di utilità utilizzata per la gestione degli errori nelle chiamate ai servizi del frontend. Ogni operazione asincrona di `noteService` e `aiService` viene eseguita tramite `serviceHandler`. Restituisce il risultato della promessa e intercetta eventuali eccezioni generate da *Axios*. In particolare, in caso di errori di rete o timeout restituisce un messaggio di connessione dedicato, mentre per gli errori HTTP propaga il messaggio ricevuto dal backend (comunemente mostrato successivamente in un toast di errore).

3.1.2.3 Composables Note & AI

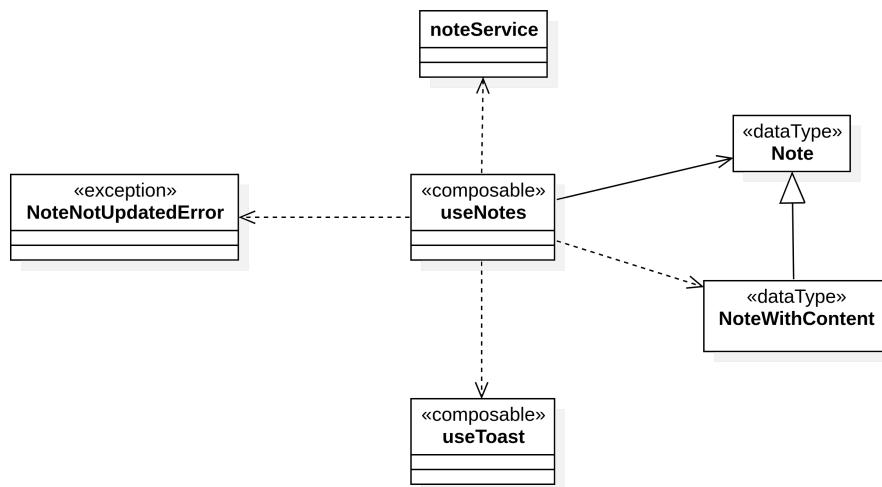


Figura 8: Diagramma UML composable useNotes

3.1.2.3.1 useNotes - Composable che incapsula la logica di gestione delle note, centralizzando lo stato, le chiamate API^G e la gestione degli errori tramite l'utilizzo di `noteService`. Segue la descrizione dei metodi esposti:

- `fetchNotes(): Promise<null>` - aggiorna l'archivio locale memorizzato in `notes`. Durante l'esecuzione abilita la variabile di stato reattiva `loading`, che viene disattivato al termine dell'operazione. Qualora `noteService` lancia un errore, memorizza il messaggio dentro la variabile reattiva `error`.
- `getNote(string $id): Promise<NoteWithContent | null>` - ritorna la nota in base all'id univoco. Qualora `noteService` lancia un errore, lo mostra in un toast di errore.



- `refreshNote(string $id): Promise<NoteWithContent | null>` - aggiorna e ritorna il metadata e il contenuto della nota in base all'id univoco, mostrando un toast di successo se l'operazione è andata a buon fine. Qualora `noteService` lancia un errore, lo mostra in un toast di errore.
 - `saveNote(NoteWithContent $note, string $content, string $name, boolean $overwrite = false): Promise<null>` - salva il nome e contenuto della nota in base all'id univoco preso dal parametro `note`. In caso il contenuto e il nome di `note` è uguale a `content` e `name` rispettivamente, e il booleano `overwrite` è settato a `false`, mostra un toast di info senza effettuare modifiche. Altrimenti:
 - In caso `note.last_edit` è diverso dal `last_edit` della versione presente in archivio (conflitto), lancia un errore di tipo `NoteNotUpdatedError`
 - Altrimenti aggiorna il nome e il contenuto della nota
- Qualora `noteService` lancia un errore, lo mostra in un toast di errore, altrimenti mostra un toast di successo.
- `renameNote(Note $note, string $newName): Promise<null>` - rinomina la nota in base all'id univoco preso dal parametro `note`. In caso il nome di `note` è uguale a `newName`, mostra un toast di info senza effettuare modifiche. Qualora `noteService` lancia un errore, lo mostra in un toast di errore, altrimenti mostra un toast di successo.
 - `removeNote(string $id): Promise<null>` - elimina una nota dall'archivio in base all'id univoco, mostrando un toast di successo se l'operazione è andata a buon fine. Qualora `noteService` lancia un errore, lo mostra in un toast di errore.
 - `storeNote(string $name, string $content): Promise<Note | null>` - archivia una nuova nota nel server locale e ritorna i metadati della nota generati, mostrando un toast di successo se l'operazione è andata a buon fine. Qualora `noteService` lancia un errore, lo mostra in un toast di errore.
 - `cloneNote(string $id, string $name): Promise<null>` - clona una nota in base all'id univoco, mostrando un toast di successo se l'operazione è andata a buon fine. Qualora `noteService` lancia un errore, lo mostra in un toast di errore.
 - `exportNote(string $id, 'pdf' | 'md' | 'html' $format): Promise<void>` - esporta la nota con identificativo `id` nel formato specificato, mostrando un toast di successo se l'operazione è andata a buon fine. Qualora `noteService` lancia un errore, lo mostra in un toast di errore.
 - `exportNoteFromRaw(string $name, string $content, 'pdf' | 'md' | 'html' $format): Promise<void>` - esporta una nota non presente nell'archivio nel formato specificato. In caso `content` sia vuoto, viene mostrato un toast di avvertenza e si interrompe l'azione. Qualora `noteService` lancia un errore, lo mostra in un toast di errore, altrimenti mostra un toast di successo.
 - `importNote(File $file): Promise<void>` - importa e archivia una nota da file, validando il formato in md e inviandolo al backend tramite `FormData`. Qualora `noteService` lancia un errore, lo mostra in un toast di errore, altrimenti mostra un toast di successo.

3.1.2.3.2 NoteNotUpdatedError Errore che estende `Error` e che indica un conflitto di salvataggio causato da una versione non aggiornata della nota.

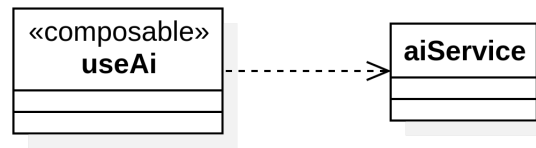


Figura 9: Diagramma UML composable useAi

3.1.2.3.3 useAi Composable responsabile^G dell'integrazione con il servizio IA tramite l'utilizzo di **aiService**. Tutta la logica asincrona è centralizzata nella funzione privata **execute**, che si occupa di gestire le variabili reattive di stato **loading**, **error** e **result** (le prime due con funzione medesima a **useNotes**, mentre l'ultima per salvare la risposta del servizio). Segue la descrizione dei metodi esposti:

- **translate(string \$content, AiLang \$lang = 'en'):** **Promise<void>** - traduce il contenuto nella lingua desiderata
- **summarize(string \$content):** **Promise<void>** - riassume il contenuto
- **rewrite(string \$content, [AiTone, ...AiTone[]] \$style):** **Promise<void>** - riscrive il contenuto in base ai toni desiderati
- **distantWriting(string \$content):** **Promise<void>** - ritorna una risposta in base al prompt inviato tramite **content**
- **blackhat(string \$content):** **Promise<void>** - critica il contenuto secondo il cappello nero
- **bluehat(string \$content):** **Promise<void>** - critica il contenuto secondo il cappello blu
- **greenhat(string \$content):** **Promise<void>** - critica il contenuto secondo il cappello verde
- **redhat(string \$content):** **Promise<void>** - critica il contenuto secondo il cappello rosso
- **whitehat(string \$content):** **Promise<void>** - critica il contenuto secondo il cappello bianco
- **yellowhat(string \$content):** **Promise<void>** - critica il contenuto secondo il cappello giallo

3.1.2.4 Pagina di archivio note

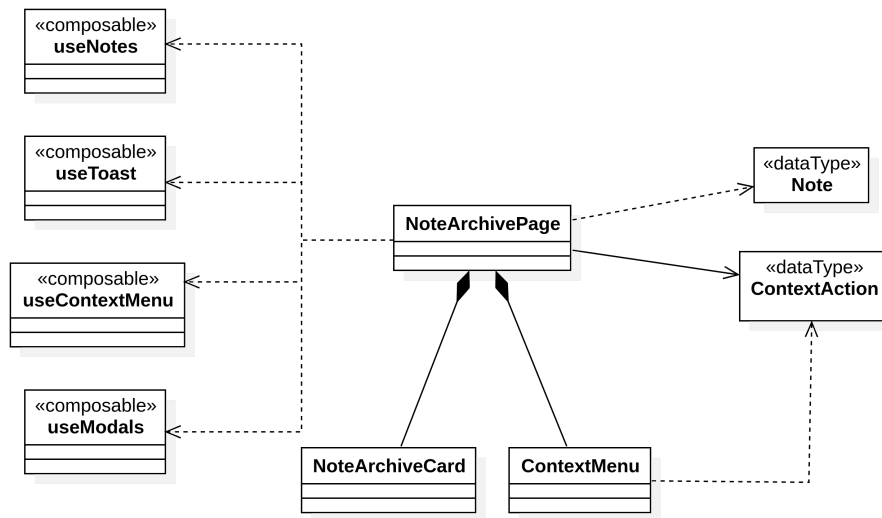


Figura 10: Diagramma UML pagina di archivio

3.1.2.4.1 NoteArchivePage Vista principale dell'archivio. Al montaggio, recupera la lista delle note tramite il composabile **useNotes**; per ogni nota l'archivio renderizza un **NoteArchiveCard**, passandogli i metadati della nota tramite **prop**. La pagina espone una barra di ricerca che filtra le note per nome in tempo reale. Dispone inoltre di un pulsante di importante che consente di caricare una nota in formato Markdown tramite un input file. Ogni **NoteArchiveCard** supporta un menu contestuale **ContextMenu** che viene chiamato dall'evento **@contextmenu**, nativo del browser. La pagina definisce le azioni del menu contestuale tramite un array di oggetti **ContextAction**: rinomina, clonazione, esportazione in vari formati ed elimina. La pagina gestisce il click di una voce del menu contestuale tramite la funzione **handleAction**, che lega l'evento **@action-clicked** di **ContextMenu** con l'azione della voce (**handler** di **ContextAction**).

3.1.2.4.2 NoteArchiveCard Componente che rappresenta una nota nell'archivio, visualizzando le informazioni principali. Quando cliccata, reindirizza tramite **<RouterLink>** alla rotta **/notes/:id** per aprirla nell'editor. Segue la descrizione dei **prop**:

- **string \$id** - id univoco della nota
- **string \$name** - nome della nota
- **string \$last_edit** - data di ultima modifica
- **string \$creation** - data di creazione

3.1.2.4.3 ContextAction (type) Tipo generico che definisce la struttura di una voce del menu contestuale. Il parametro **T** indica il tipo dell'elemento su cui l'azione opera:

- **string \$label** - nome della voce nel menu
- **(T \$item) => void | Promise<void> \$handler** - funzione opzionale sincrona / asincrona che riceve l'elemento come argomento
- **ContextActionVariant \$variant** - opzionale, determina la variante visiva della voce. (**ContextActionVariant** è un *Union Type* che può essere **'default'** o **'warning'**)



3.1.2.4.4 useContextMenu Composable responsabile^G della gestione dello stato del menu contestuale. La variabile reattiva `selectedItem` di tipo `T` rappresenta il tipo di elemento selezionato. Oltre a `selectedItem`, mantiene due stati reattivi: `isOpen` (controlla la visibilità del menu), `position` (determina le coordinate di rendering). Segue la descrizione dei metodi esposti:

- `open(MouseEvent $event, T $item)` - apre il menu contestuale alle coordinate del puntatore, passando l'elemento selezionato tramite `item`. Utilizza la funzione `preventDefault` per sopprimere il menu contestuale predefinito del browser
- `openAt(number $x, number $y, T $item)` - apre il menu contestuale alle coordinate passate come parametri
- `close()` - chiude il menu contestuale

3.1.2.4.5 ContextMenu Componente che gestisce il rendering del menu contestuale. Riceve la posizione e le voci del menu tramite `prop`. Viene montato direttamente sul `<body>` del documento tramite `Teleport`, meccanismo nativo di Vue che consente di renderizzare un componente al di fuori della gerarchia DOM del componente padre. Tramite la funzione `updateMenuPosition`, calcola la posizione definitiva del menu, attendendo che il DOM sia aggiornato prima di leggere le dimensioni reali dell'elemento (usando `nextTick`). Il componente quindi può gestire la visualizzazione correggendo automaticamente la posizione da destra a sinistra, o dall'alto al basso, mantenendo un padding minimo costante dai bordi dello schermo. Segue la descrizione dei `prop`:

- `ContextAction[] $actions` - array di voci del menu
- `number $x` - posizione x del menu
- `number $y` - posizione y del menu

Segue la descrizione degli `emit`:

- `action-clicked: [ContextAction $action]` - evento emesso quando l'utente clicca una voce del menu
- `close: []` - evento emesso quando l'utente clicca fuori dal menu contestuale (`onClickOutside`)

3.1.2.5 Pagina di editor note

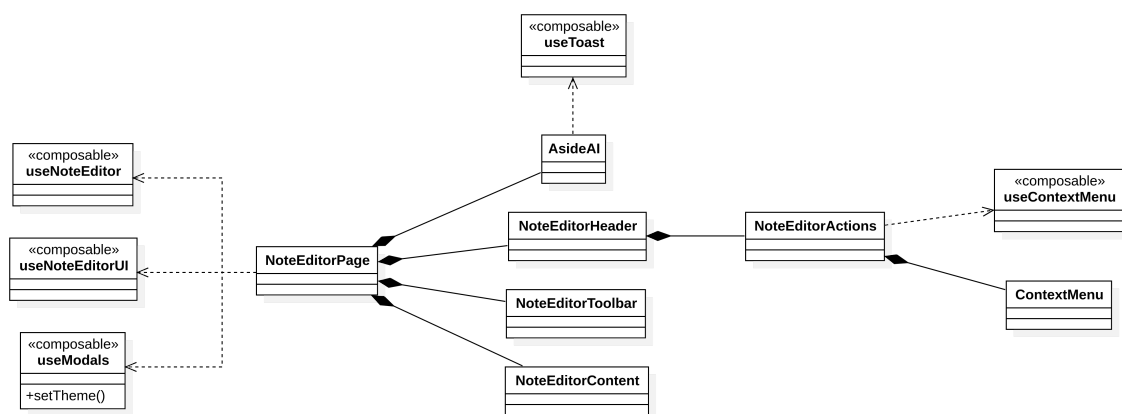


Figura 11: Diagramma UML pagina di editor

3.1.2.5.1 NoteEditorPage Vista principale dell'editor di note. Al suo interno vengono montati `NoteEditorHeader`, `NoteEditorToolbar` e `NoteEditorContent`. La pagina implementa un guard di navigazione tramite `onBeforeRouteLeave`: quando l'utente tenta di abbandonare la pagina con modifiche non salvate, viene aperto il modale `DiscardPromise`.

3.1.2.5.2 useNoteEditor Composable che gestisce lo stato e la logica della nota nell'editor. All'interno viene implementato il pattern provide / inject di Vue (`provideNoteEditor / useNoteEditor`) per rendere lo stato disponibile all'intera gerarchia di componenti figli della pagina. Lo stato centrale è composto da `noteContent` e `noteName`, che rappresentano il contenuto e il nome della nota, e da `isDirty`, che segnala la presenza di modifiche non salvate tramite un `watch` che confronta il contenuto attuale con quello originale (normalizzando prima il testo tramite `normalizeEditorHtml`). Segue la descrizione dei metodi esposti:

- `saveTheNote(boolean $exit = false)` - gestisce i diversi casi di salvataggio:
 - Primo salvataggio: controlla se l'editor non ha un id associato, e chiama la funzione privata `saveNew` per archiviare la nota nel server locale
 - Salvataggio nota esistente: salva il nome e il contenuto della nota in base all'id univoco
 - In caso di conflitto (`NoteNotUpdatedError`), apre il modale `SavePromise`: in base alla scelta dell'utente, chiama le funzioni private di sovrascrittura (`overwrite`), di salvataggio con nome (`saveAs`) o di aggiornamento (`update`)

il booleano `exit` indica se alla fine del salvataggio il composable dovrebbe cambiare rotta per aggiornare l'id all'interno della rotta (nel caso di primo salvataggio e del salvataggio con nome)

- `saveTheNoteAs(boolean $exit = false)` - gestisce il salvataggio con nome. Mostra un toast di errore in caso si tratti di una nuova nota non presente nell'archivio. Il booleano `exit` ha la stessa funzione di quello citato sopra
- `exportTheNote('pdf' | 'md' | 'html' $format)` - chiama la funzione `exportNoteFromRaw` di `useNotes` con lo stato corrente della nota
- `loadNote()` - carica la nota dall'archivio in base al parametro `id` nella rotta (`/notes/:id`) e aggiorna le variabili di stato
- `setEditorContent(string: html)` - aggiorna la variabile di stato reattiva `noteContent` con il parametro `html`

3.1.2.5.3 useNoteEditorUI Composable responsabile^G della gestione dell'interfaccia dell'editor. Si occupa esclusivamente della logica di presentazione, delegando la logica di business a `useNoteEditor` e le chiamate AI e `useAi`. Lo stato gestito comprende:

- la modalità di visualizzazione `viewMode`, che tramite l'*Union Type* `ViewMode` può essere `editor`, `split` o `render`
- lo stato del pannello AI (`isAiOpen`, `aiAction`, `selectedText`, `aiResult`)
- le opzioni di configurazione delle funzioni AI (`hatMode`, `languageMode`, `rewriteStyle`)
- il riferimento diretto all'elemento DOM dell'editor `editorRef`

Il composable gestisce il rendering Markdown tramite la libreria *marked*. La formattazione del testo è gestita da `applyFormat`, che opera sulla selezione corrente del browser tramite `document.execCommand`. Il sistema undo / redo sfrutta anch'esso `execCommand`, delegando la gestione della cronologia al browser. Segue la descrizione dei metodi esposti:

- `setViewMode(ViewMode $mode)` - aggiorna la modalità di visualizzazione dell'editor
- `setEditorRefHTMLElement $element` - salva il riferimento all'elemento HTML dell'editor
- `handleEditorInput()` - sincronizza il contenuto corrente dell'editor con lo stato dell'applicazione aggiornando il contenuto della nota
- `undoEdit()` - esegue l'operazione di undo dell'ultima modifica effettuata nell'editor



- `redoEdit()` - esegue l'operazione di redo dell'ultima modifica effettuata nell'editor
- `applyFormat('ordered_list' | 'unordered_list' | 'bold' | 'italic' | 'underline' | 'strikethrough' | 'comment' | 'link' $type)` - applica o rimuove una formattazione Markdown o HTML sul testo selezionato
- `openAiPanel('summarize' | 'hats' | 'translate' | 'rewrite' | 'distant writing' $action)` - apre il pannello delle funzionalità^G IA salvando il testo selezionato e l'azione richiesta
- `closeAiPanel()` - chiude il pannello IA e resetta l'azione selezionata
- `handleAiRun(payload): Promise<void>` - esegue l'elaborazione IA richiesta utilizzando `useAI`
- `insertAiResult(InsertMode $mode)` - inserisce nell'editor il risultato generato dall'IA secondo la modalità scelta (`InsertMode Union Type = 'before' | 'after' | 'replace' | 'bottom'`)
- `handleBeforeInput()` - intercetta le modifiche effettuate all'interno di blocchi IA e segnala eventuali cambiamenti che potrebbero invalidare contenuti generati automaticamente
- `handlePaste(ClipboardEvent $event)` - gestisce l'incollaggio del testo nell'editor
- `handleEditorKeydown(KeyboardEvent $event)` - gestisce gli eventi della tastiera nell'editor
- `stripAiMarkers(string $html): string` - rimuove dal contenuto HTML tutti i marcatori e metadati associati ai blocchi IA, restituendo solo il testo effettivo
- `retranslateAiBlock(HTMLElement $child): Promise<void>` - rigenera automaticamente una traduzione IA aggiornata quando il testo sorgente viene modificato
- `escapeHtml(string $value): string` - effettua l'escape dei caratteri HTML speciali per prevenire interpretazioni non desiderate del contenuto
- `htmlToMarkdownText(string $html): string` - converte il contenuto HTML dell'editor in testo Markdown pulito, eliminando marcatori IA e normalizzando gli spazi
- `formatList(string $text, 'ordered_list' | 'unordered_list' $type): string` - formatta un insieme di righe come lista ordinata o non ordinata
- `handleListEnter(Node $textNode, number $cursorOffset, Range $range): boolean` - gestisce automaticamente il comportamento del tasto **Enter** all'interno delle liste Markdown, continuando o terminando la lista

3.1.2.5.4 AsideAI Pannello laterale dell'intelligenza artificiale, renderizzando come elemento `<aside>` sul lato destro dello schermo. Il pannello si adatta dinamicamente all'azione IA corrente tramite due computed property: `panelTitle` e `actionLabel`, che derivano il titolo e l'etichetta del pulsante principale dell'azione selezionata. Le opzioni di configurazione vengono mostrate condizionalmente in base all'azione:

- un selettore del cappello per la modalità *Sei cappelli di De Bono*
- un selettore della lingua per la traduzione
- un gruppo di pulsanti a selezione multipla per gli stili di riscrittura (grammatica, estensione, lessico e stile)

L'esecuzione dell'azione è gestita da `runAction`, che costituisce il payload corretto in base all'azione corrente, comunicandolo alla pagina padre tramite l'evento `run`. Il risultato restituito dall'IA viene visualizzato in un'area di testo scorrevole. Segue la descrizione dei `prop`:

- `boolean $open` - indica la visibilità del pannello



- `AiAction $action` - rappresenta l'azione IA selezionata
- `string $selectedText` - contiene il testo selezionato nell'editor
- `string $result` - contiene il risultato prodotto dall'IA
- `boolean $loading` - indica se una richiesta IA è attualmente in esecuzione (per mostrare uno stato di caricamento e disabilitare i controlli)
- `HatMode $hatMode` - definisce il cappello selezionato (`HatMode Union Type = 'white' | 'red' | 'black' | 'yellow' | 'green' | 'blue' )`)
- `LanguageMode $languageMode` - rappresenta la lingua selezionata durante la traduzione (`LanguageMode Union Type = 'it' | 'en' | 'fr' | 'de' | 'es' )`)
- `RewriteStyle[] rewriteStyle` - contiene l'elenco degli stili di riscrittura selezionati (`RewriteStyle Union Type = 'grammar' | 'extension' | 'lexicon' | 'stylistic' )`)

Segue la descrizione degli `emit`:

- `close []` - emesso quando l'utente chiude il pannello
- `update:hatMode [HatMode $value]` - aggiorna il cappello selezionato
- `update:languageMode [LanguageMode $value]` - aggiorna la lingua scelta
- `update:rewriteStyle [RewriteStyle $value]` - aggiorna gli stili selezionati per la riscrittura
- `run [payload]` - richiede l'esecuzione di un'operazione IA inviando l'azione selezionata, il testo da elaborare ed eventuali opzioni aggiuntive
- `insert ['before' | 'after' | 'replace' | 'bottom' mode]` - richiede l'inserimento del risultato IA nell'editor specificando la modalità di inserimento

3.1.2.5.5 NoteEditorHeader Componente che costituisce l'intestazione dell'editor di note, strutturata in tre colonne. La prima colonna contiene un campo di input per il nome della nota, compatibile con la direttiva `v-model` di Vue, insieme all'indicatore di salvataggio per avvisare l'utente di modifiche non salvate. La colonna centrale presenta il selettore della modalità di visualizzazione (editor, split, render). La terza colonna monta il componente `NoteEditorActions`. Segue la descrizione dei `prop`:

- `string $name` - nome della nota
- `boolean $isDirty` - valore booleano che controlla se la nota ha subito modifiche
- `ViewMode $viewMode` - la modalità di visione dell'editor

Segue la descrizione degli `emit`:

- `update:name [string $value]` - emesso al cambio del nome della nota
- `change-view [ViewMode $mode]` - emesso al cambio della modalità di visualizzazione

3.1.2.5.6 NoteEditorActions Componente che raggruppa le azioni disponibili sulla nota nell'header dell'editor. Accede allo stato dell'editor tramite `useNoteEditor` sfruttando il meccanismo `provide / inject`. Espone tre controlli: il pulsante *Salva*, che chiama `saveTheNote`; il pulsante *Salva come...*, che chiama `saveTheNoteAs` e che viene disabilitato quando la nota è nuova; il pulsante *Esporta*, che apre un menu contestuale con le opzioni di esportazione, tramite l'utilizzo di un menu contestuale.



3.1.2.5.7 NoteEditorToolbar Componente che costruisce la barra degli strumenti dell'editor. Viene diviso verticalmente in tre gruppi funzionali: il primo gruppo contiene i controlli di cronologia (undo / redo), il secondo raccoglie i controlli di formattazione del testo e il terzo espone le azioni AI disponibili. Contiene inoltre un contatore di lettere / parole reattivo. Segue la descrizione dei prop:

- `number $wordCount` - numero di parole nella nota
- `number $charCount` - numero di lettere nella nota

Segue la descrizione degli emit:

- `undo: []` - emesso quando si clicca il pulsante di undo
- `redo: []` - emesso quando si clicca il pulsante di redo
- `format: ['bold' | 'italic' | 'underline' | 'strikethrough' | 'comment' | 'link' | 'ordered_list' | 'unordered_list' $type]` - emesso quando si clicca un pulsante di formattazione
- `ai: [Exclude<AiAction, null> action]` - emesso quando si clicca un pulsante di interazione con IA

3.1.2.5.8 NoteEditorContent Componente che costituisce l'area di lavoro principale dell'editor, diviso in due sezioni, sezione di editor e sezione di render. La prima sezione delega al browser la gestione nativa della modifica del testo. Al montaggio, il contenuto iniziale viene iniettato direttamente nell'`innerHTML` dell'elemento. Un `watch` sul contenuto garantisce la sincronizzazione dell'`innerHTML` quando il contenuto viene modificato esternamente (ad esempio dopo un'operazione IA). Gli eventi nativi dell'editor vengono propagati alla pagina padre, che vengono gestiti tramite `useNoteEditorUI`. La seconda sezione, l'area di rendering, visualizza il contenuto Markdown convertito in HTML tramite la direttiva `v-html`. Segue la descrizione dei prop:

- `string $content` - il contenuto Markdown della nota
- `ViewMode $viewMode` - la modalità di visione dell'editor
- `string | Promise<string> $renderHtml` - il contenuto renderizzato della nota

Segue la descrizione degli emit:

- `editor-ready: [HTMLElement $element]` - emesso al montaggio del componente, fornisce il riferimento a `HTMLElement`
- `input: []` - emesso ogni volta che il contenuto dell'editor viene modificato dall'utente
- `beforeInput: [InputEvent $event]` - intercetta l'evento `beforeInput` per permettere di gestire o validare l'input in modo preventivo
- `paste: [ClipboardEvent $event]` - emesso durante l'incollaggio di contenuti nell'editor
- `keydown: [KeyboardEvent $event]` - notifica la pressione dei tasti da tastiera
- `click: [MouseEvent $event]` - emesso al click del mouse all'interno dell'editor

3.2 Architettura di deployment

La scelta della tipologia di **architettura di deployment** è stata guidata dal contesto di utilizzo del prodotto, dai requisiti di scalabilità previsti dal capitolato^G e dalle caratteristiche dello stack tecnologico.

Non avendo ricevuto indicazioni specifiche riguardo a vincoli di deployment e non prevedendo significative espansioni o modifiche strutturali, il team ha optato per un'**architettura monolitica**.

Questa soluzione, oltre ad essere in linea con le esigenze del prodotto, semplifica e velocizza la fase di progettazione^G e sviluppo^G ed evita la complessità introdotta da un'architettura a microservizi.

Il deployment finale viene realizzato tramite un'**immagine Docker monolitica**, contenente al proprio interno tutti i componenti necessari all'esecuzione dell'applicazione: server web, runtime PHP, codice applicativo Laravel, dipendenze backend, build degli asset frontend e worker delle code. In questo modo l'applicazione può essere avviata in modo riproducibile su qualunque ambiente che supporti Docker^G, senza richiedere l'installazione manuale di Composer, Node.js, npm o altre dipendenze applicative sulla macchina ospitante.

Docker^G Compose viene utilizzato come strumento di supporto per semplificare l'avvio del container^G, la configurazione delle variabili d'ambiente, il mapping delle porte e la gestione dei volumi persistenti. Nel deploy finale non viene però adottata una suddivisione in più container^G: Docker^G Compose avvia un unico servizio applicativo, mantenendo il sistema semplice da distribuire anche su server di produzione.

3.2.1 Organizzazione del container

Nel sistema di deploy finale, l'applicazione è eseguita all'interno di un unico container^G denominato **app**. Tale container^G racchiude le responsabilità tecniche necessarie all'erogazione del servizio, pur mantenendo l'architettura applicativa monolitica.

- **app**
 - Contiene l'applicazione Laravel;
 - contiene le dipendenze PHP installate tramite Composer durante la fase di build dell'immagine;
 - contiene gli asset frontend compilati tramite Vite durante la fase di build dell'immagine;
 - utilizza Nginx come web server e punto di ingresso per le richieste HTTP;
 - utilizza PHP-FPM per l'esecuzione del codice PHP dell'applicazione;
 - esegue il worker delle code Laravel;
 - utilizza Supervisor per gestire i processi interni al container^G;
 - espone l'applicazione sulla porta interna 8080, mappabile su una porta esterna configurabile;
 - utilizza SQLite come sistema di persistenza dei dati applicativi.

3.2.2 Persistenza dei dati

La persistenza dei dati è gestita tramite volumi Docker^G dedicati. Nel progetto^G attuale le note vengono salvate come file testuali nello storage locale di Laravel, mentre il database SQLite viene utilizzato solo per le tabelle infrastrutturali previste dalle migration Laravel, come sessioni, cache e code.

- Le note e gli altri file applicativi vengono preservati tramite un volume Docker^G associato alla cartella:

`/var/www/html/storage`

- Il database SQLite viene creato automaticamente nel percorso interno:

`/data/database.sqlite`

- Il percorso `/data` viene associato a un volume Docker^G persistente;
- Il database SQLite non contiene le note, ma dati infrastrutturali Laravel, come sessioni, cache e code;
- Le migration Laravel vengono eseguite automaticamente all'avvio del container.



3.2.3 Build degli asset frontend

Gli asset frontend non vengono compilati in fase di esecuzione del container. Durante la build dell'immagine Docker^G viene eseguita una fase dedicata alla compilazione del frontend tramite Vite.

- Le dipendenze JavaScript vengono installate durante la build;
- Vite produce gli asset statici ottimizzati per l'ambiente di produzione;
- gli asset generati vengono copiati nell'immagine finale e serviti da Nginx;
- non è necessario eseguire un dev server Vite nell'ambiente di produzione.

Il container^G **vite** rimane quindi utile esclusivamente in contesti di sviluppo^G, mentre non fa parte dell'architettura di deploy finale.

3.2.4 Configurazione e avvio

La configurazione del deploy avviene tramite variabili d'ambiente, definite nel file `.env` posto nella root del progetto^G o direttamente nell'ambiente di esecuzione del container.

Il comando principale di avvio è:

```
make deploy
```

Tale comando costruisce l'immagine Docker^G monolitica e avvia il container^G tramite il file `docker-compose.deploy.yml`. Il sistema di avvio si occupa automaticamente di:

- creare il file di configurazione Laravel all'interno del container^G;
- generare una chiave applicativa, se non già configurata;
- creare il database SQLite, se non presente;
- eseguire le migration;
- preparare le cache applicative;
- avviare Nginx, PHP-FPM e il worker delle code Laravel.

La porta esterna può essere configurata tramite la variabile `APP_PORT`; internamente il container^G espone il servizio sulla porta 8080. Ad esempio, impostando:

```
APP_PORT=8090
APP_URL=http://localhost:8090
```

l'applicazione sarà accessibile sulla porta 8090, mentre il servizio interno continuerà a utilizzare la porta 8080.

3.2.5 Container non presenti nel deploy finale

Rispetto all'ambiente di sviluppo^G o a precedenti configurazioni, il deploy finale non prevede i seguenti container^G separati:

- **web**: le funzionalità^G di web server sono incluse nel container^G **app** tramite Nginx;
- **db**: PostgreSQL non viene avviato;
- **vite**: la compilazione degli asset frontend avviene durante la build dell'immagine Docker^G e non tramite un dev server in produzione.

Questa organizzazione consente di ottenere un deploy più semplice e portabile, mantenendo un unico container^G applicativo e riducendo le dipendenze operative richieste all'ambiente ospitante.

4 Stato dei requisiti funzionali

Requisito	Descrizione	Rispettato
RF-O-1	Il sistema deve permettere all'utente di visualizzare l'archivio delle note presenti e per ogni nota il suo nome, la data di creazione e la data dell'ultima modifica	SI
RF-O-2	Il sistema deve permettere all'utente di aprire una nota	SI
RF-O-3	Il sistema deve permettere all'utente di uscire da una nota	SI
RF-O-4	Il sistema deve permettere all'utente di creare una nuova nota	SI
RF-O-5	Il sistema deve permettere all'utente di salvare la nota su cui sta lavorando	SI
RF-O-6	Il sistema deve permettere all'utente di salvare una nota non ancora presente nell'archivio per la prima volta	SI
RF-O-7	Il sistema deve permettere all'utente di clonare una nota	SI
RF-O-8	Alla clonazione di una nota, il sistema deve assegnare automaticamente un nome di default (es. "Nota <i>data-ora</i> ") che l'utente può modificare	SI
RF-O-9	Il nome di default generato nella clonazione di una nota deve essere univoco all'interno dell'archivio e facilmente riconoscibile all'utente	SI
RF-O-10	Il sistema deve permettere all'utente di ricercare le note per nome nell'archivio	SI
RF-O-11	Il sistema deve permettere all'utente di eliminare una nota	SI
RF-O-12	Il sistema deve permettere all'utente di rinominare una nota	SI
RF-O-13	Il sistema deve permettere all'utente di esportare una nota aperta	SI
RF-O-14	Il sistema deve permettere all'utente di esportare una nota direttamente dall'archivio	SI
RF-O-15	L'esportazione deve preservare integralmente il contenuto testuale della nota, senza perdita o alterazione di caratteri	SI
RF-O-16	L'esportazione deve preservare correttamente i caratteri Unicode del testo (es. accenti e simboli), senza sostituzioni o corruzione	SI
RF-O-17	Al termine dell'esportazione, il sistema deve notificare esplicitamente l'errore di esportazione all'utente	SI
RF-O-18	Il sistema deve permettere all'utente di importare una nota	SI
RF-O-19	L'importazione deve preservare integralmente il contenuto testuale della nota, senza perdita o alterazione di caratteri (incl. Unicode)	SI
RF-O-20	In caso di file non valido o corrotto, il sistema deve interrompere l'importazione senza creare note parziali e deve notificare chiaramente l'errore all'utente	SI

RF-O-21	Il sistema deve permettere all'utente di scrivere nell'editor con caratteri Unicode (es. lettere accentate e simboli)	SI
RF-O-22	Il sistema deve compilare la sintassi Markdown	SI
RF-O-23	Il sistema deve permettere la visualizzazione del documento formattato secondo la sintassi Markdown	SI
RF-O-24	L'aggiornamento della visualizzazione formattata deve avvenire in un tempo adeguatamente veloce	SI
RF-O-25	Il sistema deve permettere di cambiare modalità di visualizzazione	SI
RF-O-26	Il sistema deve fornire un'area di editing di testo in cui è possibile inserire e modificare del testo per la compilazione Markdown	SI
RF-O-27	Il sistema deve permettere l'applicazione di formattazioni Markdown senza scrivere manualmente la sintassi	SI
RF-O-28	L'editor deve rispondere all'input e alla formattazione dell'utente in modo fluido, senza ritardi percepibili durante la digitazione	SI
RF-O-29	Il sistema deve permettere l'applicazione e la rimozione della formattazione testo grassetto senza scrivere manualmente la sintassi	SI
RF-O-30	Il sistema deve permettere l'applicazione e la rimozione della formattazione testo corsivo senza scrivere manualmente la sintassi	SI
RF-O-31	Il sistema deve permettere l'applicazione e la rimozione della formattazione testo sottolineato senza scrivere manualmente la sintassi	SI
RF-O-32	Il sistema deve consentire l'applicazione e la rimozione della formattazione testo barrato senza richiedere l'inserimento manuale della sintassi	SI
RF-O-33	Il sistema deve consentire l'inserimento e la rimozione di link ipertestuali senza richiedere l'inserimento manuale della sintassi	SI
RF-O-34	L'editor deve rendere i link ipertestuali facilmente riconoscibili (es. evidenziandoli graficamente) e distinguibili dal testo normale	SI
RF-O-35	Il sistema deve consentire l'applicazione e la rimozione della formattazione testo commentato senza richiedere l'inserimento manuale della sintassi	SI
RF-O-36	Il sistema deve consentire l'inserimento di elenchi numerati	SI
RF-O-37	Il sistema deve consentire l'inserimento di elenchi puntati	SI
RF-O-38	Il sistema deve permettere all'utente di annullare l'ultima modifica fatta	SI
RF-O-39	Il sistema deve permettere all'utente di ripristinare la modifica precedentemente annullata	SI
RF-O-40	Il sistema deve permettere all'utente di selezionare porzioni di testo, sia tramite mouse che tastiera	SI
RF-O-41	Il sistema deve permettere all'utente di copiare il testo selezionato	SI
RF-O-42	Il sistema deve permettere all'utente di tagliare il testo selezionato	SI

RF-O-43	L'editor deve poter fare copiare e tagliare all'utente il testo in modo fluido, senza ritardi percepibili	SI
RF-O-44	L'editor deve preservare correttamente il contenuto e la struttura del testo copiato e tagliato, senza perdita o alterazioni di caratteri	SI
RF-O-45	Il sistema deve permettere all'utente di incollare del testo nell'area di testo	SI
RF-O-46	L'editor deve poter fare incollare all'utente il testo in modo fluido, senza ritardi percepibili	SI
RF-O-47	L'editor deve preservare e incollare correttamente il contenuto e la struttura del testo, senza perdita o alterazioni di caratteri	SI
RF-O-48	Il sistema deve permettere all'utente di esportare una nota aperta in MD	SI
RF-O-49	Il sistema deve permettere all'utente di esportare una nota direttamente dall'archivio in MD	SI
RF-O-50	Il sistema deve avvisare l'utente della presenza di una nota con lo stesso nome in seguito al primo salvataggio	SI
RF-O-51	Il sistema deve permettere all'utente di criticare il testo secondo il modello dei "Sei Cappelli"	SI
RF-O-52	Il sistema deve permettere all'utente di criticare il testo con una "Visione Emotiva" (Cappello Rosso)	SI
RF-O-53	Il sistema deve permettere all'utente di criticare il testo con una "Visione Neutra" (Cappello Bianco)	SI
RF-O-54	Il sistema deve permettere all'utente di criticare il testo con una "Visione Ottimista" (Cappello Giallo)	SI
RF-O-55	Il sistema deve permettere all'utente di criticare il testo con una "Visione dell'avvocato del Diavolo" (Cappello Nero)	SI
RF-O-56	Il sistema deve permettere all'utente di criticare il testo con una "Visione Originale" (Cappello Verde)	SI
RF-O-57	Il sistema deve permettere all'utente di criticare il testo con una "Visione Logica e Strutturata" (Cappello Blu)	SI
RF-O-58	Il sistema deve permettere all'utente di riassumere del testo tramite un agente esterno	SI
RF-O-59	Il sistema deve permettere all'utente di riscrivere del testo tramite un agente esterno	SI
RF-O-60	Il sistema deve permettere all'utente di scegliere il modo in cui verrà riscritto il testo (correzione grammaticale, estensione del testo, miglioramento del lessico, variazione stilistica)	SI
RF-O-61	Il sistema deve permettere all'utente di tradurre del testo tramite un agente esterno	SI
RF-O-62	Il sistema deve permettere l'invio da parte dell'utente di un prompt a un agente esterno per la generazione di testo (Distant Writing)	SI
RF-O-63	Il sistema deve permettere all'utente di sostituire nell'editor il testo originale con quello generato dall'agente esterno (riassunto, riscrittura, traduzione)	SI

RF-O-64	Il sistema deve permettere all'utente di aggiungere nell'editor, in seguito alla selezione del testo originale, quello generato dall'agente esterno (riassunto, riscrittura, traduzione)	SI
RF-O-65	Il sistema deve permettere all'utente di aggiungere nell'editor, in coda a tutto il testo presente nell'editor, quello generato dall'agente esterno (riassunto, riscrittura, traduzione)	SI
RF-O-66	Il sistema deve permettere all'utente di "rifiutare" il testo generato dall'agente esterno (riassunto, riscrittura, traduzione)	SI
RF-O-67	Il sistema deve permettere all'utente di copiare negli appunti il testo generato dall'agente esterno (riassunti, riscrittura, traduzione, generazione)	SI
RF-O-68	L'agente esterno deve produrre testo grammaticalmente, sintatticamente e semanticamente corretto	SI
RF-O-69	Il sistema deve avvisare l'utente in caso dell'impossibilità nell'eseguire una richiesta	SI
RF-O-70	Il sistema deve commentare il testo selezionato dall'utente utilizzato per interrogare la generazione (Distant Writing)	SI
RF-O-71	Se l'utente aggiunge una traduzione a una parte del testo selezionato, il sistema deve avvisarlo ogni volta che il testo tradotto viene modificato	SI
RF-O-72	Il sistema deve consentire all'utente di aggiornare una traduzione precedentemente inserita se il testo originale è presente ed è stato modificato	SI
RF-O-73	Il sistema deve consentire all'utente di rimuovere una traduzione precedentemente inserita	SI
RF-O-74	Il sistema deve consentire all'utente di rimuovere il testo originale di una traduzione precedentemente inserita	SI
RF-O-75	Il sistema deve notificare all'utente la perdita del testo originale di riferimento qualora venga eliminato, lasciando la traduzione priva di collegamento	SI
RF-O-76	Il sistema non deve consentire l'inserimento di una traduzione all'interno di una porzione di testo già tradotta	SI
RF-O-77	L'agente deve produrre traduzioni corrette nella lingua selezionata	SI
RF-O-78	Il sistema alla chiusura di una nota, se presenti delle modifiche non salvate deve avvisare l'utente della presenza di modifiche	SI
RF-O-79	Il sistema in caso di problemi di connessione con l'agente esterno deve avvisare l'utente	SI
RF-O-80	Durante una richiesta alla LLM, il sistema deve fornire all'utente un feedback chiaro sullo stato dell'operazione (es. indicatore di caricamento/elaborazione) fino al completamento o alla segnalazione di errore	SI
RF-O-81	Il feedback sullo stato della comunicazione con la LLM deve essere non bloccante e deve evitare che l'utente avvii richieste duplicate inconsapevolmente	SI



RF-O-82	Il sistema deve permettere all'utente di visualizzare l'editor con il contenuto della nota dopo l'apertura di questa	SI
RF-O-83	Il sistema deve avvisare l'utente in caso di tentativo di apertura di una nota non più esistente	SI
RF-O-84	Il sistema deve avvisare l'utente in caso di fallimento dell'apertura di una nota a causa di problemi di connessione al server	SI
RF-O-85	In caso di tentativo di chiusura di una nota con modifiche non salvate, il sistema deve avvisare l'utente e chiedere se desidera salvare le modifiche prima di chiudere	SI
RF-O-86	Il sistema deve permettere all'utente di salvare le modifiche effettuate all'interno di una nota	SI
RF-O-87	Il sistema deve avvisare l'utente in caso di presenza di una versione più recente della nota aperta nell'archivio	SI
RF-O-88	Il sistema deve permettere all'utente di sovrascrivere la nota presente nell'archivio con la versione aperta in caso di conflitto di versioni	SI
RF-O-89	Il sistema deve permettere all'utente di salvare la nota aperta con un nuovo nome in caso di conflitto con la versione presente nell'archivio	SI
RF-O-90	Il sistema deve permettere all'utente di aggiornare la nota aperta con le modifiche presenti nella versione più recente nell'archivio in caso di conflitto di versioni	SI
RF-O-91	Il sistema deve avvisare l'utente in caso di errori durante il salvataggio della nota causati da problemi di connessione al server	SI
RF-O-92	Il sistema deve avvisare l'utente in caso di tentativo di clonazione di una nota non più esistente	SI
RF-O-93	Il sistema deve avvisare in caso di impossibilità di clonare una nota a causa di problemi di connessione al server	SI
RF-O-94	Il sistema deve chiedere all'utente di confermare l'eliminazione di una nota	SI
RF-O-95	Il sistema deve avvisare l'utente in caso di impossibilità di eliminazione di una nota a causa di problemi di connessione al server	SI
RF-O-96	Il sistema deve impedire all'utente di rinominare una nota con un nome già presente nell'archivio e visualizzare un messaggio di errore	SI
RF-O-97	Il sistema deve avvisare l'utente in caso di impossibilità di rinominare una nota a causa di problemi di connessione al server	SI
RF-O-98	Il sistema deve avvisare l'utente in caso di impossibilità di esportare una nota a causa di problemi di connessione al server	SI
RF-O-99	Il sistema deve dare la possibilità all'utente di importare una nota in formato MD	SI
RF-O-100	Il sistema deve avvisare l'utente in caso di problemi di connessione al server durante l'importazione di una nota	SI

RF-O-101	Il sistema deve permettere all'utente di inserire elementi in un elenco puntato o numerato	SI
RF-O-102	Il sistema deve permettere all'utente di rimuovere elementi da un elenco puntato o numerato	SI
RF-O-103	Il sistema deve avvisare l'utente in caso di problemi di connessione con l'agente esterno durante la ritraduzione di una porzione di testo precedentemente tradotta	SI
RF-D-1	Il sistema deve aggiornare automaticamente i risultati della ricerca ad ogni modifica dell'input di ricerca	SI
RF-D-2	Il sistema deve permettere all'utente di esportare una nota aperta in PDF	SI
RF-D-3	Il sistema deve permettere all'utente di esportare una nota aperta in HTML	SI
RF-D-4	Il sistema deve permettere all'utente di esportare una nota direttamente dall'archivio in PDF	SI
RF-D-5	Il sistema deve permettere all'utente di esportare una nota direttamente dall'archivio in HTML	SI
RF-D-6	Il sistema deve permettere all'utente di utilizzare scorciatoie da tastiera per formattare il testo	NO
RF-D-7	Il sistema deve comunicare all'utente eventuali errori grammaticali nell'editor	NO
RF-D-8	Il sistema deve consentire all'utente di eseguire un minimo di 20 operazioni consecutive di annullamento e di ripristino delle modifiche sull'editor della nota corrente	NO
RF-D-9	In caso di incollaggio di contenuti non testuali, l'editor deve mantenere la reattività e mostrare una notifica non bloccante che spieghi l'azione effettuata (rifiuto o conversione in testo)	NO
RF-D-10	L'archivio deve aggiornarsi dinamicamente con i risultati della ricerca ad ogni modifica dell'input con una risposta percepita come immediata dall'utente	SI
RF-D-11	L'editor deve evidenziare in modo chiaro e distinguibile la porzione di testo utilizzato come prompt per l'interrogazione del LLM, così da renderla immediatamente riconoscibile all'utente	SI
RF-D-12	L'aggiornamento della visualizzazione formattata non deve bloccare l'input dell'utente durante la digitazione	SI
RF-OP-1	Il sistema deve aggiornare automaticamente la visualizzazione del testo formattato ad ogni modifica del Markdown, senza richiedere un'azione esplicita	SI
RF-OP-2	Il sistema deve comunicare all'utente eventuali errori di sintassi nei marcatori nell'editor	NO
RF-OP-3	L'editor deve permettere all'utente di interagire direttamente con la LLM tramite marcatori	NO
RF-OP-4	Il sistema deve permettere all'utente di registrarsi tramite nome utente e password	NO
RF-OP-5	Se password o nome utente non sono corrette il sistema non deve permettere la registrazione	NO
RF-OP-6	Il sistema deve permettere all'utente di accedere al proprio account	NO

RF-OP-7	Se password o nome utente non sono corrette il sistema non deve permettere l'autenticazione	NO
RF-OP-8	L'utente può effettuare il logout e terminare la sessione	NO
RF-OP-9	Il sistema deve permettere ad un utente autenticato di eliminare il proprio account	NO
RF-OP-10	Il sistema deve richiedere la password dell'utente per l'eliminazione del suo account	NO
RF-OP-11	Il sistema deve avvisare l'utente che la password inserita per l'eliminazione dell'account è sbagliata	NO
RF-OP-12	Il sistema deve richiedere la conferma all'utente per l'eliminazione del suo account	NO
RF-OP-13	Il sistema deve permettere all'utente autenticato di salvare note nel proprio account	NO
RF-OP-14	Il sistema deve permettere all'utente autenticato di aprire una delle note salvate nel proprio account	NO
RF-OP-15	Il sistema durante il salvataggio, nel caso in cui il DataBase non sia raggiungibile, deve visualizzare un messaggio d'errore	NO
RF-OP-16	Il sistema durante l'apertura di una nota, nel caso in cui il DataBase non sia raggiungibile, deve visualizzare un messaggio d'errore	NO
RF-OP-17	Le credenziali dell'utente devono essere trasmesse in modo sicuro durante l'autenticazione e la registrazione	NO
RF-OP-18	Il sistema deve completare una richiesta di autenticazione e registrazione in un breve lasso di tempo	NO
RF-OP-19	Il sistema non deve memorizzare né mostrare password in chiaro	NO
RF-OP-20	Il sistema deve limitare ad un massimo di 3 tentativi di accesso falliti per ridurre il rischio di attacchi a forza bruta	NO
RF-OP-21	In caso di credenziali errate, il messaggio di errore non deve rivelare se sia il nome utente o la password ad essere errata	NO
RF-OP-22	I messaggi di errore in autenticazione e registrazione devono essere chiari e indicare all'utente come correggere l'input	NO
RF-OP-23	I campi di inserimento credenziali devono fornire feedback immediato su validità del formato (es. vincoli su nome utente/password), senza attendere l'invio del form	NO
RF-OP-24	Dopo l'autenticazione, la sessione deve scadere automaticamente dopo un periodo di inattività pari a 1 mese	NO
RF-OP-25	Il sistema deve avvisare l'utente in caso di mancata registrazione/autenticazione dovuta a problemi di connessione al server	NO
RF-OP-26	Il sistema deve avvisare l'utente in caso di impossibilità di eliminazione dell'account a causa di problemi di connessione al server	NO

Tabella Requisiti Funzionali